



`alittlebyte.swe@gmail.com`

Specifica Tecnica

Gruppo	aLittleByte (gruppo 19)
Versione	1.0.0
Redattori	–
Verificatori	–
Destinatari	Prof. Tullio Vardanega, Prof. Riccardo Cardin
Data	–

Registro delle Modifiche

Versione	Data	Descrizione	Redattore	Verificatore	Approvatore
----------	------	-------------	-----------	--------------	-------------

Indice

1	Introduzione	4
1.1	Contenuto del documento	4
1.2	Struttura del documento	4
1.3	Glossario	4
1.4	Riferimenti	5
1.4.1	Riferimenti normativi	5
1.4.2	Riferimenti informativi	5
2	Tecnologie	5
2.1	Linguaggi di programmazione	6
2.2	Framework	6
2.3	Librerie	6
2.4	Contratto API	7
2.5	Ambiente locale e provisioning	8
2.6	Edge e web server	9
2.7	Servizi cloud AWS	10
2.8	Tecnologie per la persistenza dei dati	11
2.9	Osservabilità	11
2.10	CI e quality gate	12
3	API	14
3.1	State API	14
3.2	Communications API	15
3.3	Documents API	16
4	Architettura del Sistema	17
4.1	Contesto di Sistema	17
4.2	Architettura Applicativa	18
4.3	Architettura Logica Interna	18
4.3.1	Frontend: Interfaccia Utente	18
4.3.2	Backend: Architettura Esagonale	18
4.3.2.1	Adapter primari	18
4.3.2.2	Dominio e applicazione	19
4.3.2.3	Adapter secondari	19
4.3.2.4	Esempio: il dominio Documents	19
4.4	Dettaglio dei Componenti	19
4.4.0.1	Documents	20
4.4.0.2	Communications	20
4.5	Architettura di Deployment	20
4.6	Containerizzazione con Docker	21
4.7	Stile architetturale	21

5	Descrizione architettura (Backend)	22
5.1	Modulo Documents (AI Co-Pilot)	22
5.1.1	Diagramma di sequenza: caricamento ed elaborazione di un documento	22
5.1.2	DocumentController	22
5.1.3	DocumentPreviewController, SendMessageController	23
5.1.4	DocumentWorkflowTaskHandler	24
5.1.5	OriginalDocument	24
5.1.6	SubDocument	25
5.1.7	UploadDocumentService	25
5.1.8	StartDocumentWorkflowService	26
5.1.9	RunOcrService	26
5.1.10	ProcessDocumentService	27
5.1.11	ExtractSubDocumentFieldsService	27
5.1.12	FinalizeDocumentWorkflowService	28
5.1.13	PollDocumentProgressService	28
5.1.14	ReviewDocumentService	29
5.1.15	PreviewDocumentService	29
5.1.16	SendMessageService	29
5.1.17	ListDocumentsService, DeleteDocumentService	30
5.1.18	EloquentDocumentRepository	31
5.1.19	Adapter AI e OCR: TextractOcrAdapter, BedrockDocumentAiAdapter	32
5.1.20	FlysystemDocumentStorageAdapter, DompdfSendMessageRenderer	32
5.1.21	LaravelDocumentEventDispatcher	33
5.2	Modulo Communications (AI Assistant Generativo)	33
5.2.1	Diagramma di sequenza: generazione di una comunicazione	33
5.2.2	CommunicationController	34
5.2.3	CommunicationWorkflowTaskHandler	35
5.2.4	Communication	35
5.2.5	GenerateCommunicationService	36
5.2.6	StartCommunicationWorkflowService	37
5.2.7	GenerateCommunicationTextService, GenerateCommunicationCoverService	37
5.2.8	FinalizeCommunicationService	38
5.2.9	PollCommunicationProgressService	38
5.2.10	CommunicationDraftService	38
5.2.11	RateCommunicationService	39
5.2.12	UpdateCommunicationCoverService	39
5.2.13	DownloadCommunicationCoverService, ExportCommunicationService	40
5.2.14	ListCommunicationsService, DeleteCommunicationService	40
5.2.15	PromptConfigurationService	41
5.2.16	EloquentCommunicationRepository	41
5.2.17	EloquentPromptConfigurationRepository	42
5.2.18	BedrockCommunicationAiAdapter	42

5.2.19	FlysystemCommunicationCoverAdapter, DompdfCommunicationPdfRenderer	43
5.2.20	LaravelCommunicationEventDispatcher	43
5.3	Classi condivise	43
6	Architettura Frontend	45
6.1	Principi di progettazione	45
6.2	Il pattern MVVM	45
6.2.1	Esempio nel progetto: AssistantPageViewModel	47
6.2.2	Esempio nel progetto: MvpStateStore (stato condiviso fra i ViewModel)	48
6.3	Definizione dei modelli di dati	49
6.3.1	Esempio nel progetto: assistant.model.ts	49
6.3.2	Esempio nel progetto: tipi del modulo Co-Pilot	49
6.4	Servizi di comunicazione con il backend	50
6.4.1	Esempio nel progetto: AssistantService	50
6.4.2	Esempio nel progetto: DocumentWorkflowService	51
6.4.3	Esempio nel progetto: SseClient	52
6.5	Serializzazione e deserializzazione dei dati	52
6.6	Micro componenti UI	52
6.6.1	Esempio nel progetto: ButtonComponent	53
6.6.2	AlertComponent, ErrorStateComponent	53
6.6.3	EmptyStateComponent, LoadingStateComponent, StatusBadgeComponent	53
6.6.4	MetricCardComponent, MetricsPanelComponent	54
6.6.5	ProgressBarComponent	54
6.6.6	SelectFieldComponent, TextAreaFieldComponent	55
6.6.7	FileDropzoneComponent	55
6.7	Macro componenti UI (pagine)	55
6.7.1	Esempio nel progetto: AssistantPage	56
6.7.2	Esempio nel progetto: CopilotPage	56
6.7.3	Esempio nel progetto: OverviewPage	57
7	Design Pattern	57
7.1	Design Pattern Backend	58
7.1.1	Adapter	58
7.1.2	Strategy	59
7.1.3	Facade	59
7.1.4	Factory Method	59
7.1.5	Singleton	60
7.1.6	Observer	60
7.1.7	Command	60
7.1.8	Pattern nativi del framework	61
7.2	Design Pattern Frontend	61
7.2.1	Facade	61
7.2.2	Presentation Model	62

1 Introduzione

1.1 Contenuto del documento

Il presente documento costituisce la Specifica Tecnica del sistema sviluppato dal gruppo *aLittleByte* per il proponente Eggon S.r.l. nell'ambito del progetto Nexum.

Al suo interno vengono illustrate le tecnologie selezionate, il contratto delle API REST esposte dal backend e le scelte architetturali adottate nella realizzazione del prodotto. Il documento ha lo scopo di fornire una visione organica e dettagliata della struttura del sistema, delle sue componenti e delle interazioni tra esse.

Deve essere inteso come riferimento tecnico per le attività di progettazione e codifica, garantendo la coerenza tra le decisioni implementative adottate e quanto emerso nella fase di Proof of Concept.

1.2 Struttura del documento

La struttura del presente documento è articolata nelle seguenti sezioni principali:

- **Sezione 1 - Introduzione:** Presenta il contenuto e gli obiettivi del documento, il glossario dei termini e i riferimenti normativi e informativi adottati.
- **Sezione 2 - Tecnologie:** Descrive lo stack tecnologico adottato, suddiviso per categoria: linguaggi, framework, librerie, ambiente locale, edge, servizi cloud AWS, persistenza dei dati, osservabilità e CI/quality gate.
- **Sezione 3 - API:** Descrive il contratto REST esposto dal backend, organizzato per area funzionale (stato applicativo, AI Assistant Generativo, AI Co-Pilot).
- **Sezione 4 - Architettura del Sistema:** Descrive il contesto di sistema, l'architettura applicativa e logica interna del backend, il dettaglio dei componenti, il deployment, la containerizzazione e lo stile architetturale complessivo.
- **Sezione 5 - Descrizione architettura (Backend):** Scende al livello delle singole classi dei due domini di prodotto (Documents, Communications): entità di dominio, casi d'uso applicativi e adapter secondari.
- **Sezione 6 - Architettura Frontend:** Descrive i principi di progettazione della SPA Angular, il pattern MVVM adottato, i modelli di dati, i servizi di comunicazione col backend e i componenti UI, micro e macro.

1.3 Glossario

Al fine di evitare ambiguità e garantire una comprensione uniforme della terminologia utilizzata, i termini tecnici, gli acronimi e le parole con un significato specifico nel contesto del progetto sono contrassegnati nel testo con una “G” ad apice (es. parola^G). La loro definizione completa è consultabile nel documento esterno *Glossario*.

1.4 Riferimenti

1.4.1 Riferimenti normativi

- **Capitolato d'appalto C5 - NEXUM (Eggon S.r.l.):**
<https://www.math.unipd.it/~tullio/IS-1/2025/Progetto/C5.pdf>
Data Ultimo accesso: 09/07/2026
- **Norme di Progetto v1.0.0:**
Documento interno del gruppo *aLittleByte* che definisce le regole, i ruoli e le procedure operative adottate nel progetto.
<https://alittlebyte-19.github.io/Documentazione/RTB/Norme%20di%20Progetto/Norme%20di%20Progetto.pdf>
Data Ultimo accesso: 09/07/2026
- **Regolamento del Progetto Didattico:**
Anno accademico 2025/2026, Corso di Ingegneria del Software, Università di Padova.
<https://www.math.unipd.it/~tullio/IS-1/2025/Dispense/PD1.pdf>
Data Ultimo accesso: 09/07/2026

1.4.2 Riferimenti informativi

- **Glossario v2.0.0:**
Documento interno del gruppo *aLittleByte* contenente le definizioni dei termini tecnici e degli acronimi utilizzati nella documentazione.
<https://alittlebyte-19.github.io/Documentazione/RTB/Glossario/Glossario.pdf>
Data Ultimo accesso: 09/07/2026
- **Analisi dei Requisiti v2.0.0:**
Documento interno del gruppo *aLittleByte* contenente la descrizione dei requisiti funzionali e non funzionali del sistema.
<https://alittlebyte-19.github.io/Documentazione/RTB/Analisi%20dei%20Requisiti/Analisi%20dei%20Requisiti.pdf>
Data Ultimo accesso: 09/07/2026

2 Tecnologie

Il presente capitolo illustra lo stack tecnologico adottato per la progettazione e la realizzazione del sistema Nexum. Le scelte implementative sono state guidate dai requisiti di vincolo definiti nell'Analisi dei Requisiti e dalle indicazioni del proponente Eggon S.r.l. Lo stack è organizzato attorno a cinque macro-aree principali: il runtime applicativo (backend Laravel su PHP e frontend Angular in TypeScript), l'infrastruttura cloud AWS per l'elaborazione intelligente dei documenti e la gestione asincrona dei job, la persistenza dei dati, lo stack di osservabilità e la pipeline di integrazione continua con quality gate automatici. L'ambiente locale replica fedelmente la configurazione di produzione attraverso la

containerizzazione con Docker e l'emulazione dei servizi cloud con LocalStack, garantendo riproducibilità e coerenza tra i diversi ambienti di sviluppo e produzione.

2.1 Linguaggi di programmazione

Nome	Versione	Descrizione
PHP	8.4	Linguaggio di scripting server-side utilizzato per la realizzazione del backend tramite il framework Laravel. La versione 8.4 introduce miglioramenti significativi alle performance e nuove funzionalità del linguaggio.
TypeScript	5.9	Superset tipizzato di JavaScript sviluppato da Microsoft, impiegato per la realizzazione del frontend Angular. La tipizzazione statica riduce gli errori a tempo di compilazione e migliora la manutenibilità del codice.
SQL	SQL:2023	Linguaggio dichiarativo per la definizione e la manipolazione dei dati nel database relazionale PostgreSQL, utilizzato per la gestione della persistenza strutturata del sistema.

Tabella 2: Linguaggi di programmazione

2.2 Framework

Nome	Versione	Descrizione
Laravel	12	Framework PHP open source per lo sviluppo di applicazioni web server-side. Fornisce un'architettura MVC, un sistema di routing, un ORM (Eloquent) e strumenti integrati per la gestione dei job asincroni. È stato scelto per la maturità del suo ecosistema rispetto ad alternative come Symfony.
Angular	21	Framework open source sviluppato da Google per la creazione di Single Page Application (SPA). Utilizza TypeScript e adotta un'architettura basata su componenti con supporto nativo per dependency injection, routing, reattive forms e signals. È stato preferito per la struttura integrata e opinata che garantisce coerenza e manutenibilità nel tempo.

Tabella 3: Framework

2.3 Librerie

Nome	Versione	Ambito	Descrizione
RxJS	7.8	Frontend	Libreria per la programmazione reattiva basata su Observable. Utilizzata per la gestione dello stato asincrono e degli stream di eventi all'interno della SPA Angular, in combinazione con i signals nativi di Angular.
@lucide/angular	1.21	Frontend	Libreria di icone open source integrata nel frontend Angular. Fornisce un set di icone SVG consistente e accessibile per l'interfaccia utente.
AWS SDK for PHP	3.380	Backend	SDK ufficiale AWS per PHP. Utilizzata dal backend Laravel per invocare Bedrock, Textract, S3, SQS, Step Functions, SSM Parameter Store e Secrets Manager tramite un'unica libreria client.
Flysystem S3 (league/flysystem-aws-s3-v3)	3.32	Backend	Adapter S3 per il filesystem astratto di Laravel. Consente di trattare i bucket S3 (o LocalStack) come un disco Laravel standard, cambiando solo la configurazione tra ambienti.
Opis JSON Schema	2.6	Backend	Libreria per la validazione di payload JSON contro uno schema. Utilizzata per validare la struttura dei dati restituiti dai modelli Bedrock durante l'estrazione dei documenti.
FPDI	2.6	Backend	Libreria PHP per l'importazione di pagine da documenti PDF esistenti. Utilizzata nella pipeline documentale per estrarre pagine specifiche da PDF massivi prima dello split.
FPDF	1.8	Backend	Libreria PHP per la generazione di documenti PDF. Utilizzata in combinazione con FPDI per produrre i sotto-documenti PDF per range di pagine a partire dai file originali.

Tabella 4: Librerie

2.4 Contratto API

Il frontend non conosce gli endpoint del backend in modo statico: il contratto OpenAPI 3.1 costituisce la fonte di verità condivisa da cui viene generato automaticamente il client TypeScript tipizzato, mantenendo frontend e backend allineati durante l'evoluzione del sistema.

Nome	Versione	Descrizione
OpenAPI	3.1	Specifica standard per la descrizione delle API REST. Costituisce il contratto condiviso tra frontend e backend e rappresenta la fonte di verità da cui vengono derivati client e documentazione.
Orval	8.23	Strumento di code generation che, a partire dalla specifica OpenAPI 3.1, genera automaticamente il client TypeScript tipizzato per il frontend Angular, eliminando la scrittura manuale delle chiamate HTTP.
Redocly CLI	latest	Strumento per la validazione e il linting del contratto OpenAPI, eseguito in pipeline sempre nell'ultima versione disponibile. Verifica la conformità della specifica agli standard e individua inconsistenze prima che vengano propagate al codice generato.

Tabella 5: Strumenti per il contratto API

2.5 Ambiente locale e provisioning

L'ambiente locale è progettato in modo che ogni membro del team lavori sulla stessa identica configurazione, vicina a quella di produzione, senza dover creare account cloud o installare servizi manualmente. I servizi AWS vengono emulati in locale tramite LocalStack, mentre l'infrastruttura è descritta come codice con Terraform, rendendola versionata e ricreabile.

Nome	Versione	Descrizione
Docker	—	Piattaforma di containerizzazione utilizzata per costruire le immagini dell'applicazione e del web server. Garantisce che l'ambiente di esecuzione sia identico per tutti i membri del team e coerente con la produzione.
Docker Compose	—	Strumento per l'orchestrazione dei container locali. Avvia l'intera infrastruttura (applicazione, database, servizi di supporto) su reti separate con un singolo comando.
Make	—	Automazione dei comandi ricorrenti tramite Makefile. Garantisce che le procedure di setup, avvio e build siano identiche per tutti i membri del team.
Composer	—	Gestore delle dipendenze PHP. Gestisce le librerie del backend e le relative versioni, garantendo riproducibilità delle build.
Node.js	22	Runtime JavaScript utilizzato dagli strumenti di build e sviluppo del frontend (Angular CLI, npm, Orval, ESLint, Jest). Eseguito in un container dedicato, non richiede un'installazione locale sulla macchina dello sviluppatore.

Nome	Versione	Descrizione
LocalStack	4.5	Emulatore locale dei servizi AWS. Riproduce in locale S3, SQS, Step Functions, SSM, Secrets Manager e KMS, consentendo di testare il flusso completo senza dipendere da risorse cloud attive o credenziali reali.
Terraform	1.10.5	Strumento di Infrastructure as Code (IaC) per il provisioning dell'infrastruttura. Sostituisce gli script manuali con una descrizione dichiarativa e versionata, ricreabile a comando tramite terraform apply .
AWS SSM Parameter Store	—	Servizio AWS per la gestione della configurazione runtime non sensibile. I parametri vengono letti dall'applicazione al bootstrap, mantenendo la configurazione fuori dal codice sorgente.
AWS Secrets Manager	—	Servizio AWS per la gestione dei segreti applicativi (credenziali, chiavi API). Mantiene i dati sensibili separati dal codice e dalla configurazione.

Tabella 6: Strumenti per l'ambiente locale e il provisioning

2.6 Edge e web server

Una richiesta HTTPS attraversa diversi livelli prima di raggiungere il codice applicativo. Traefik opera come entry point TLS esterno; in locale un secondo Nginx (*edge-cdn*) emula il ruolo di una CDN, servendo la SPA Angular direttamente da S3/LocalStack e inoltrando le richieste `/api`, `/health` e `/ready` all'Nginx applicativo, che a sua volta fa da proxy verso PHP-FPM. In produzione il ruolo di CDN è coperto da AWS CloudFront, mentre l'Nginx applicativo resta un'immagine dedicata esclusivamente al proxy verso PHP-FPM.

Nome	Versione	Descrizione
Traefik	3.4	Reverse proxy posizionato davanti a tutti i servizi. Si occupa della terminazione TLS, dello smistamento delle richieste in base all'host e dell'esposizione delle metriche per il sistema di osservabilità.
Nginx (applicativo)	1.27	Web server utilizzato come proxy per le chiamate <code>/api</code> verso PHP-FPM tramite FastCGI. Immagine di produzione, buildata e scansionata, che non conosce LocalStack.
Nginx (edge-cdn)	1.29	Secondo Nginx, presente solo in locale, che emula il ruolo di una CDN/edge davanti a S3-LocalStack per servire gli asset statici della SPA Angular. In produzione questo ruolo è sostituito da AWS CloudFront.
PHP-FPM	8.4	Runtime PHP con processo FastCGI. Esegue il codice dell'applicazione Laravel. È stato preferito a FrankenPHP e Laravel Octane per un comportamento più prevedibile e una gestione dei processi più semplice in fase di PoC.

Nome	Versione	Descrizione
------	----------	-------------

Tabella 7: Edge e web server

2.7 Servizi cloud AWS

Le funzionalità di intelligenza artificiale, l'elaborazione asincrona dei documenti e la protezione dei dati si basano su servizi gestiti AWS. L'adozione di servizi gestiti elimina la necessità di infrastrutture dedicate, garantendo scalabilità automatica e alta disponibilità. In locale tutti questi servizi vengono emulati da LocalStack.

Nome	Versione	Descrizione
Amazon Bedrock	—	Servizio AWS per l'accesso a modelli linguistici di grandi dimensioni (LLM). Di default utilizza Amazon Nova Lite per la generazione dei testi e Stability SD3.5 Large per le immagini di copertina nell'AI Assistant Generativo, oltre che per la classificazione e lo split automatico dei documenti nell'AI Co-Pilot.
Amazon Textract	—	Servizio AWS di riconoscimento ottico dei caratteri (OCR). Estrae testo e dati strutturati da documenti scansionati o immagini, leggendo i file direttamente da S3.
AWS Step Functions	—	Servizio di orchestrazione delle pipeline asincrone. Due state machine distinte coordinano rispettivamente la pipeline documentale (OCR, estrazione AI, persistenza risultati) e la pipeline delle comunicazioni (generazione testo/copertina), gestendo tentativi, timeout e stato tramite task token. È stato preferito a soluzioni self-hosted come Temporal o Camunda.
Amazon SQS	—	Servizio di code di messaggi gestite. Sono presenti due code separate, una per la pipeline documentale e una per quella delle comunicazioni, scelte per la loro durabilità e per la gestione automatica dei messaggi falliti. Un Worker dedicato per pipeline consuma i messaggi dalla propria coda e risponde all'orchestratore Step Functions con un token di callback.
Amazon SQS DLQ	—	Dead Letter Queue: una coda secondaria SQS per pipeline che raccoglie automaticamente i messaggi che hanno superato il numero massimo di tentativi (<code>maxReceiveCount</code>), consentendo l'analisi e il reindirizzamento dei task falliti.

Nome	Versione	Descrizione
Amazon S3	—	Servizio di object storage. Un bucket dedicato archivia i documenti originali, i sotto-documenti generati dalla pipeline e le copertine delle comunicazioni, con oggetti cifrati at-rest tramite KMS; un secondo bucket ospita gli asset statici della SPA Angular, serviti dall'edge locale o da CloudFront in produzione. L'accesso a entrambi è regolato da policy IAM.
AWS KMS	—	Servizio di gestione delle chiavi crittografiche. Utilizzato per la cifratura at-rest degli oggetti del bucket documentale, garantendo la protezione dei documenti sensibili.
AWS IAM	—	Servizio di gestione delle identità e degli accessi. Definisce i ruoli di esecuzione per Step Functions e le policy di accesso alle risorse, seguendo il principio del minimo privilegio.

Tabella 8: Servizi cloud AWS

2.8 Tecnologie per la persistenza dei dati

I dati del sistema hanno nature diverse e vengono gestiti con strumenti dedicati: PostgreSQL per la persistenza strutturata e transazionale, Redis per lo stato volatile ad accesso rapido.

Nome	Versione	Descrizione
PostgreSQL	16	Sistema di gestione di database relazionali open source, erogato come servizio gestito in cloud. Conserva i dati strutturati dell'applicazione (documenti, estratti, audit, stato dei task) e, grazie al supporto JSONB, gestisce anche payload semi-strutturati senza cambiare database. È stato preferito a MySQL e a soluzioni documentali come MongoDB per affidabilità e flessibilità.
Redis	7	Data store in-memory open source, erogato come servizio gestito in cloud. Gestisce la cache applicativa, le sessioni utente e il rate limiting, sempre con scadenza. Non viene utilizzato come coda della pipeline, ruolo affidato ad Amazon SQS.

Tabella 9: Tecnologie per la persistenza dei dati

2.9 Osservabilità

Il sistema è strumentato con OpenTelemetry, uno standard indipendente dal fornitore che consente di raccogliere metriche, tracce e log senza vincolarsi a prodotti specifici. I log applicativi vengono inviati via OTLP al Collector, mentre le metriche sono esposte

dall'applicazione su un endpoint dedicato (`/internal/metrics`) e raccolte dal Collector tramite scraping; entrambi i segnali vengono poi smistati verso i rispettivi backend di storage, mentre Grafana li correla in cruscotti unificati.

Nome	Versione	Descrizione
OpenTelemetry (OTel)	—	Standard open source per la strumentazione delle applicazioni. Definisce il formato OTLP con cui log (e potenzialmente tracce) vengono inviati al Collector, consentendo di cambiare backend di osservabilità senza modificare il codice applicativo.
OTel Collector	0.153.0	Componente centrale che riceve i log via OTLP e le metriche tramite scraping (Nginx, Traefik e se stesso), smistandoli verso Prometheus (metriche), Tempo (tracce) e Loki (log).
Prometheus	3.9.0	Sistema di raccolta e storage delle metriche. Raccoglie le metriche esposte dai servizi tramite scrape e definisce le regole su cui Alertmanager genera gli avvisi.
Grafana Tempo	2.9.0	Backend per le tracce distribuite. Riceve le tracce dal Collector e consente la correlazione con metriche e log all'interno di Grafana.
Grafana Loki	3.3.2	Sistema di aggregazione dei log. Riceve i log dal Collector via OTLP e da Alloy, indicizzandoli per consentire ricerche contestuali.
Grafana Alloy	1.5.1	Agente per la raccolta dei log dai container Docker Compose. Invia i log di container a Loki tramite pipeline di raccolta.
Grafana	12.3.0	Piattaforma di visualizzazione. Correla metriche, tracce e log in cruscotti unificati, consentendo l'analisi contestuale degli eventi senza dover ricorrere a stack separati come ELK o Jaeger.
Alertmanager	0.29.0	Componente per la gestione e l'instradamento degli alert. Genera avvisi a partire da regole definite su Prometheus, legate a eventi concreti del dominio applicativo.

Tabella 10: Stack di osservabilità

2.10 CI e quality gate

Ogni modifica inviata al repository attiva la pipeline di integrazione continua su GitHub Actions, che ricostruisce le immagini e le pubblica su GHCR solo dopo che tutti i controlli automatici sono stati superati. I quality gate coprono stile del codice, analisi statica, test, sicurezza delle dipendenze e accessibilità.

Nome	Versione	Categoria	Descrizione
GitHub Actions	—	CI/CD	Orchestratore della pipeline CI. Attivato da push e pull request, esegue in parallelo le pipeline di backend, frontend e stack.
GHCR	—	CI/CD	GitHub Container Registry: registro in cui vengono pubblicate le immagini Docker solo dopo il superamento di tutti i quality gate.
Docker Hub	—	CI/CD	Utilizzato esclusivamente per il pull autenticato delle immagini base, al fine di evitare i limiti di rate anonimo. Non è il registro di pubblicazione delle immagini prodotte.
Trivy	0.66.0	Sicurezza	Scanner di vulnerabilità per immagini Docker. Analizza le immagini costruite (vulnerabilità, secret, configurazione) prima della pubblicazione su GHCR.
npm audit	—	Sicurezza	Analisi delle vulnerabilità nelle dipendenze npm di produzione del frontend Angular.
Pint	1.29	Qualità backend	Formattatore e linter per il codice PHP, basato sulle convenzioni di Laravel.
PHPStan / Larastan	3.10	Qualità backend	Strumento di analisi statica per PHP con estensione specifica per Laravel. Individua errori di tipo e problemi di logica senza eseguire il codice.
Pest	4.7	Qualità backend	Framework di testing per PHP utilizzato per i test unitari e di integrazione del backend Laravel.
ESLint	9.39	Qualità frontend	Lint per il codice TypeScript della SPA Angular. Verifica la conformità alle regole di stile e individua potenziali errori.
Jest	30.4	Qualità frontend	Framework di testing JavaScript utilizzato per i test unitari del frontend Angular, in combinazione con TypeScript.
Redocly + Orval diff	8.23	Contratto API	Redocly (sempre ultima versione) valida il contratto OpenAPI e Orval rigenera il client tipizzato; un diff verifica che il client generato sia allineato con il contratto aggiornato.

Nome	Versione	Categoria	Descrizione
terraform validate	1.10.5	IaC	Valida formato e correttezza sintattica/semantica della configurazione Terraform prima di qualsiasi applicazione.
axe	4.13	Accessibilità	Strumento di testing automatico per l'accessibilità (a11y) dell'interfaccia utente, integrato nella pipeline CI.
pally	9.1	Accessibilità	Strumento complementare ad axe per il testing dell'accessibilità. Esegue test a11y sulle pagine dell'applicazione.
Playwright	1.60	Accessibilità	Framework per l'automazione di un browser reale (Chromium). Utilizzato per eseguire gli audit di accessibilità con axe e Pally e per verificare che la SPA funzioni correttamente con la Content Security Policy applicata.

Tabella 11: Strumenti per CI e quality gate

3 API

Il sistema espone un contratto REST versionato (`/api/v1`), definito tramite la specifica OpenAPI 3.1 in `openapi/v1/alittlebyte-mvp-api.yaml` e consumato dalla SPA Angular tramite il client generato con Orval. Ogni richiesta è autenticata tramite l'header `X-Mvp-User-Id` (schema `MvpIdentity`), che veicola le identity claims iniettate dal confine aziendale o dal middleware locale della MVP.

Il rate limiting è granulare per gruppo di endpoint: 60 richieste/minuto di default, 20/minuto per le operazioni di generazione e scrittura più onerose (creazione bozze, rigenerazione, upload OCR, aggiornamento copertina, salvataggio configurazioni di prompt), 30/minuto per anteprima, export e valutazione, 120/minuto per la sola lettura della copertina già generata. Gli endpoint che restituiscono un PDF (anteprima ed export delle comunicazioni) supportano la cache HTTP tramite `ETag/If-None-Match`, evitando di rigenerare il documento se il client possiede già l'ultima versione.

L'avanzamento delle elaborazioni asincrone (generazione contenuti, pipeline documentale) è notificato alla SPA tramite endpoint di streaming Server-Sent Events (SSE), evitando il polling. Le liste di comunicazioni e documenti supportano filtri lato server e paginazione. Le API sono organizzate in tre aree funzionali, corrispondenti ai tag della specifica OpenAPI: lo stato applicativo, l'AI Assistant Generativo (comunicazioni) e l'AI Co-Pilot (documenti).

3.1 State API

Espone lo stato corrente dei due moduli applicativi, utilizzato dalla SPA per popolare le viste principali senza richieste multiple.

Endpoint	Metodo	Descrizione
/api/v1/state	GET	Restituisce in un'unica risposta le metriche e lo storico delle comunicazioni dell'AI Assistant Generativo, insieme alle metriche e ai sotto-documenti dell'AI Co-Pilot.

Tabella 12: State API

3.2 Communications API

Gestione del ciclo di vita completo di una comunicazione generata dall'AI Assistant Generativo: richiesta della bozza, revisione e modifica, rigenerazione, salvataggio o scarto, valutazione, esportazione e gestione della copertina. Include inoltre la gestione delle configurazioni di prompt salvate per il riuso.

Endpoint	Metodo	Descrizione
/api/v1/communications	GET	Lista delle bozze di comunicazione del tenant, con filtri opzionali (parola chiave nel prompt, tono, stile, data) e paginazione; esclude le bozze scartate.
/api/v1/communications	POST	Richiede una bozza di comunicazione a partire da prompt, tono e stile; avvia in modo asincrono la pipeline delle comunicazioni e restituisce l'identificativo della comunicazione e l'URL di streaming.
/api/v1/communications/{communication}	PUT	Aggiorna titolo e corpo di una bozza di comunicazione.
/api/v1/communications/{communication}	DELETE	Elimina definitivamente una comunicazione dallo storico.
/api/v1/communications/{communication}/regenerate	POST	Rigenera la bozza di comunicazione mantenendo o aggiornando i parametri di prompt, tono e stile.
/api/v1/communications/{communication}/save	POST	Salva la bozza nello storico, lasciandola comunque modificabile e rigenerabile.
/api/v1/communications/{communication}/discard	POST	Scarta la bozza di comunicazione.
/api/v1/communications/{communication}/favorite	POST	Aggiunge la comunicazione ai preferiti.
/api/v1/communications/{communication}/favorite	DELETE	Rimuove la comunicazione dai preferiti.

Endpoint	Metodo	Descrizione
/api/v1/communications/{communication}/rating	POST	Assegna una valutazione da 1 a 5 stelle alla bozza generata; valutazioni successive per la stessa generazione vengono rifiutate.
/api/v1/communications/{communication}/stream	GET	Endpoint SSE che notifica alla SPA l'avanzamento della generazione (testo e copertina) per una comunicazione specifica.
/api/v1/communications/{communication}/cover-image	GET	Download dell'immagine di copertina generata per la comunicazione (PNG, JPEG o WebP).
/api/v1/communications/{communication}/cover-image	POST	Sostituisce manualmente l'immagine di copertina generata dall'AI, caricando un file.
/api/v1/communications/{communication}/cover-image	DELETE	Rimuove l'immagine di copertina associata alla comunicazione.
/api/v1/communications/{communication}/preview	GET	Anteprima del PDF impaginato della comunicazione, con supporto ETag per evitare rigenerazioni non necessarie.
/api/v1/communications/{communication}/export	GET	Download del PDF finale della comunicazione come allegato, con lo stesso supporto ETag dell'anteprima.
/api/v1/prompt-configurations	POST	Salva la configurazione corrente di prompt (tono, stile, testo) per un riutilizzo successivo.
/api/v1/prompt-configurations/{promptConfiguration}	DELETE	Elimina definitivamente una configurazione di prompt salvata.

Tabella 13: Communications API

3.3 Documents API

Gestione del ciclo di vita dei documenti caricati dall'AI Co-Pilot, della relativa pipeline di elaborazione asincrona (OCR, estrazione, revisione) e della composizione del messaggio di invio verso il destinatario individuato.

Endpoint	Metodo	Descrizione
/api/v1/documents	GET	Lista dei sotto-documenti del tenant, con filtri opzionali (ricerca su nome/cognome/azienda, stato di invio, soglia e criterio di confidenza, mese/anno) e paginazione.

Endpoint	Metodo	Descrizione
/api/v1/documents/ocr	POST	Carica un documento e avvia in modo asincrono la pipeline documentale (OCR tramite Textract, estrazione e classificazione tramite Bedrock); restituisce l'URL di streaming per seguirne l'avanzamento.
/api/v1/documents/{originalDocument}/stream	GET	Endpoint SSE che notifica alla SPA l'avanzamento dell'elaborazione per un documento originale specifico.
/api/v1/documents/{subDocument}	DELETE	Elimina un sotto-documento già elaborato dalla pipeline.
/api/v1/documents/{subDocument}/extracted-data	PUT	Corregge i dati estratti automaticamente per un sotto-documento (dati anagrafici, tipo documento, data, descrizione, punteggio di confidenza).
/api/v1/documents/{subDocument}/review	POST	Segna un sotto-documento come validato manualmente, ad esempio a valle di una revisione human-in-the-loop per i casi a bassa confidenza.
/api/v1/documents/{subDocument}/preview	GET	Restituisce l'anteprima in formato PDF del sotto-documento.
/api/v1/documents/{subDocument}/send-preview	GET	Anteprima PDF del messaggio di invio precompilato per il sotto-documento.
/api/v1/documents/{subDocument}/send-export	GET	Download del messaggio di invio precompilato come PDF allegato.
/api/v1/documents/{subDocument}/send-message	PUT	Corregge i campi del messaggio di invio precompilato per il sotto-documento.

Tabella 14: Documents API

4 Architettura del Sistema

Questo capitolo descrive l'architettura del sistema, dal generale al particolare: contesto, architettura applicativa, architettura logica interna del backend, dettaglio dei componenti, deployment, containerizzazione e stile architetturale complessivo.

4.1 Contesto di Sistema

L'utente finale interagisce con la piattaforma tramite browser, via HTTPS. Il sistema delega l'elaborazione intelligente dei contenuti a due servizi AWS esterni: l'AI Co-Pilot invia i documenti caricati ad AWS Textract per il riconoscimento ottico dei caratteri; l'AI Assistant Generativo invia i prompt ad AWS Bedrock per la generazione di testi e

immagini. Le integrazioni con AWS avvengono esclusivamente lato backend; il browser comunica solo con il sistema.

4.2 Architettura Applicativa

Il sistema si compone dei seguenti container principali. Il frontend è una Single Page Application in Angular 21. Il backend è un'API in Laravel 12 su PHP 8.4, servita da PHP-FPM e Nginx, che espone esclusivamente un contratto REST in formato JSON, senza rendering lato server. Le operazioni pesanti — OCR e invocazione dei modelli generativi — sono delegate in modo asincrono ad AWS Step Functions e Amazon SQS, consumate da un processo Worker separato dall'API. La persistenza è affidata a PostgreSQL per i dati applicativi e a Redis per cache, sessioni e rate limiting; nessuno dei due è utilizzato come coda, ruolo riservato a SQS.

4.3 Architettura Logica Interna

Il frontend è dedicato all'interfaccia utente. Il backend è organizzato per dominio applicativo secondo i principi dell'architettura esagonale (*ports & adapters*).

4.3.1 Frontend: Interfaccia Utente

Sviluppato in Angular come Single Page Application, gestisce l'interfaccia e l'interazione con l'utente finale.

- Gestisce gli input dell'utente (caricamento documenti, richieste di generazione, revisione dei dati estratti).
- Comunica con il backend tramite chiamate API RESTful, generate a partire dal contratto OpenAPI condiviso.
- Riceve l'avanzamento delle elaborazioni asincrone tramite Server-Sent Events, senza polling.

4.3.2 Backend: Architettura Esagonale

Il backend adotta il pattern *ports & adapters*, applicato ai due domini di prodotto: **Documents** (AI Co-Pilot) e **Communications** (AI Assistant Generativo). La logica di business è disaccoppiata dai dettagli tecnici — framework, database, SDK esterni — per garantire manutenibilità, testabilità e sostituibilità dei singoli componenti. Il backend si articola in quattro parti.

4.3.2.1 Adapter primari Ricevono uno stimolo esterno e lo traducono in una chiamata a un caso d'uso, senza contenere regole di business. Nella forma più comune sono i Controller HTTP. Il gestore dei task di workflow è un adapter primario equivalente, guidato da un messaggio SQS o da una callback di Step Functions invece che da una richiesta HTTP.

4.3.2.2 Dominio e applicazione Il dominio contiene le regole di business e le porte che definiscono le capacità del sistema, senza dipendenze da Eloquent, da un client AWS o da HTTP. Le tre aggregazioni principali (**SubDocument**, **OriginalDocument**, **Communication**) sono entità di dominio a sé stanti, distinte dai model Eloquent: incapsulano le proprie transizioni di stato e le relative regole (es. una copertina non può essere generata prima del testo) invece di lasciarle implicite nei singoli servizi applicativi. L'applicazione implementa le porte primarie orchestrando dominio ed entità esclusivamente attraverso le porte secondarie.

4.3.2.3 Adapter secondari Implementano la comunicazione con i sistemi esterni: il database PostgreSQL tramite repository Eloquent, i modelli AI tramite i wrapper di Bedrock e Textract, lo storage documentale tramite Flysystem e S3.

4.3.2.4 Esempio: il dominio Documents Un Controller riceve l'upload, lo valida e delega l'esecuzione al caso d'uso corrispondente, che salva il file e avvia la pipeline asincrona senza conoscere i dettagli di Step Functions. Il Worker raggiunge lo stesso dominio applicativo tramite un adapter primario diverso, guidato dal workflow, e delega ai casi d'uso di estrazione e revisione. Il risultato è persistito tramite il repository del dominio; un evento dedicato notifica audit e metriche. Il dominio Communications segue lo stesso schema.

4.4 Dettaglio dei Componenti

La tabella seguente quantifica la composizione interna dei due domini: casi d'uso, porte verso l'esterno, eventi di dominio e adapter primari.

Dominio	Casi d'uso	Porte secondarie	Eventi (= listener)	Adapter primario
Documents	12	6	14	4 Controller HTTP + WorkflowTaskHandler
Communications	14	6	21	6 Controller HTTP + WorkflowTaskHandler

Tabella 15: Metriche strutturali dei due domini esagonali

I due domini condividono tre porte secondarie di infrastruttura, non specifiche di dominio: una che astrae l'avvio di un'esecuzione Step Functions, una per la generazione di identificativi univoci e una per i confini transazionali sulle scritture DB pure. Ciascuna è implementata da un solo adapter condiviso, bindato una volta sola nel service provider. La conformità alla Dependency Rule è verificata automaticamente: uno script, eseguito a ogni build della pipeline CI, analizza il codice delle due cartelle di dominio e interrompe la build in presenza di un riferimento a Illuminate, all'SDK AWS, a un modello Eloquent o al namespace dell'altro dominio — i due domini non si conoscono a vicenda, solo Support, Identity, Workflow, Audit e Observability sono infrastruttura condivisa legittima.

4.4.0.1 Documents

- **Ingresso:** UploadDocumentUseCase.
- **Orchestrazione pipeline:** StartDocumentWorkflowUseCase, RunOcrUseCase, ProcessDocumentUseCase, ExtractSubDocumentFieldsUseCase, FinalizeDocumentWorkflowUseCase.
- **Consultazione e gestione:** ListDocumentsUseCase, DeleteDocumentUseCase, PreviewDocumentUseCase, PollDocumentProgressUseCase.
- **Revisione umana:** ReviewDocumentUseCase.
- **Invio:** SendMessageUseCase.
- **Porte secondarie:** DocumentRepository (persistenza), OcrGatewayPort (Textract), DocumentAiGatewayPort (Bedrock), DocumentStoragePort (S3/Flysystem), SendMessageRendererPort (rendering PDF del messaggio di invio), DocumentEventDispatcherPort (pubblicazione degli eventi di dominio).

4.4.0.2 Communications

- **Generazione:** GenerateCommunicationUseCase, GenerateCommunicationTextUseCase, GenerateCommunicationCoverUseCase.
- **Orchestrazione pipeline:** StartCommunicationWorkflowUseCase, FinalizeCommunicationUseCase, PollCommunicationProgressUseCase.
- **Gestione della bozza:** CommunicationDraftUseCase, UpdateCommunicationCoverUseCase, DownloadCommunicationCoverUseCase, DeleteCommunicationUseCase.
- **Consultazione, esportazione, valutazione:** ListCommunicationsUseCase, ExportCommunicationUseCase, RateCommunicationUseCase.
- **Configurazioni di prompt riutilizzabili:** PromptConfigurationUseCase.
- **Porte secondarie:** CommunicationRepository (persistenza), CommunicationAiGatewayPort (Bedrock), CommunicationCoverStoragePort (S3/Flysystem per le copertine), CommunicationPdfRendererPort (rendering PDF), CommunicationEventDispatcherPort (pubblicazione degli eventi di dominio), PromptConfigurationRepository (persistenza delle configurazioni salvate).

4.5 Architettura di Deployment

Il backend e la SPA frontend sono distribuiti come container separati, raggiunti da un unico entry point TLS. Una richiesta attraversa Traefik, l'edge (locale o CloudFront in produzione) e l'Nginx applicativo prima di raggiungere PHP-FPM, dove i Controller traducono la richiesta in una chiamata al caso d'uso corrispondente. Un secondo processo,

il Worker, esegue lo stesso dominio applicativo raggiunto tramite i messaggi SQS e le callback di Step Functions invece che da HTTP: API e Worker sono due adapter primari sullo stesso nucleo applicativo, non due sistemi separati.

In locale l'intera topologia — orchestrazione Step Functions/SQS inclusa — è riprodotta con Docker Compose, LocalStack e Terraform, con gli stessi componenti usati in produzione.

4.6 Containerizzazione con Docker

L'intero stack è containerizzato; i dettagli di ciascun servizio sono descritti nella Sezione 2. I container principali sono:

- **Frontend (SPA):** esegue la build statica Angular, servita dall'edge; nessuna logica di business.
- **Backend (API):** esegue PHP-FPM dietro Nginx, gestisce le richieste HTTP verso il dominio applicativo.
- **Worker:** esegue lo stesso dominio applicativo del backend, in ascolto su Step Functions/SQS anziché su HTTP.
- **Database (PostgreSQL) e Cache (Redis):** persistenza applicativa e stato volatile, su volumi Docker dedicati.
- **Emulazione AWS (LocalStack) e provisioning (Terraform):** riproducono in locale, senza credenziali reali, l'infrastruttura di produzione.

4.7 Stile architetturale

Lo stile architetturale del backend è un monolite modulare organizzato per dominio: un nucleo esagonale limitato ai due domini di prodotto, circondato da un'infrastruttura condivisa che non vi è assoggettata. Tre caratteristiche lo definiscono:

- **Testabilità del dominio senza framework:** i casi d'uso di Documents e Communications sono istanziabili con `new`, senza container, database o LocalStack; la suite di test dedicata usa implementazioni in memoria delle porte secondarie al posto di quelle reali.
- **Orchestrazione asincrona esternalizzata:** le operazioni onerose non transitano dal sistema di code nativo di Laravel, ma da Step Functions e SQS; l'adapter che le riceve è trattato allo stesso livello di un Controller HTTP.
- **Conformità verificata automaticamente:** la Dependency Rule è un controllo eseguito a ogni build, non una convenzione documentata.

Il perimetro dell'esagonale resta limitato per scelta esplicita: Identity, Audit, Observability, Support e l'infrastruttura condivisa di Workflow restano organizzati per servizio, perché nessuno di essi ha oggi — né è previsto abbia nell'arco della MVP — un'implementazione alternativa da rendere sostituibile.

5 Descrizione architettura (Backend)

Questo capitolo scende al livello delle singole classi che compongono il nucleo dei due domini di prodotto, completando la vista d'insieme del capitolo precedente. Per ciascun modulo sono descritti l'adapter primario, le entità di dominio, tutti i casi d'uso applicativi e gli adapter secondari, seguendo lo stesso ordine in cui intervengono nella pipeline descritta dal diagramma di sequenza; la categorizzazione dei casi d'uso per responsabilità resta quella della Sezione 4.4.

5.1 Modulo Documents (AI Co-Pilot)

Il modulo Documents è la componente del sistema dedicata alla gestione e all'elaborazione intelligente della documentazione HR caricata dai consulenti del lavoro. Il suo obiettivo è ridurre il lavoro manuale di classificazione e smistamento dei documenti: un file caricato viene automaticamente suddiviso, riconosciuto tramite OCR e associato al destinatario corretto, lasciando all'operatore solo la revisione dei casi a bassa confidenza. Il modulo si integra con Amazon Textract per il riconoscimento ottico dei caratteri e con Amazon Bedrock per la classificazione e l'estrazione dei dati, orchestrando l'intera pipeline in modo asincrono tramite AWS Step Functions.

5.1.1 Diagramma di sequenza: caricamento ed elaborazione di un documento

Il diagramma descrive la sequenza delle operazioni che avvengono quando un consulente del lavoro carica un documento tramite l'endpoint dedicato. Il `DocumentController` riceve la richiesta, delega il salvataggio del file a `UploadDocumentService` e, subito dopo, l'avvio della pipeline a `StartDocumentWorkflowUseCase`; la risposta HTTP torna al client con l'URL di streaming, senza attendere l'elaborazione.

Da quel momento la sequenza prosegue in modo asincrono. Il Worker riceve dalla state machine Step Functions un task alla volta e lo smista al caso d'uso corrispondente, un passo per fase della pipeline: riconoscimento ottico, classificazione, estrazione dei campi per ogni destinatario individuato, infine persistenza dei risultati e pubblicazione dell'evento di completamento.

In termini di classi, quello smistamento passa per `DocumentWorkflowTaskHandler`, che delega rispettivamente a `RunOcrService` (`textract.ocr`), `ProcessDocumentService`, `ExtractSubDocumentFieldsService` (`bedrock.extract`) e `FinalizeDocumentWorkflowUseCase` (`persist.results`, `dispatch.domain_event`). Ogni passo aggiorna lo stato di `OriginalDocument` tramite `EloquentDocumentRepository`.

In parallelo, la SPA resta connessa all'endpoint SSE esposto da `DocumentController::stream()`, che interroga `PollDocumentProgressUseCase` a ogni iterazione e inoltra al client i sotto-documenti man mano che vengono estratti, fino all'evento finale di completamento o di errore.

5.1.2 DocumentController

Adapter primario HTTP del dominio: riceve le richieste, le traduce in chiamate ai casi d'uso tramite le rispettive porte primarie e restituisce la risposta al client. Non contiene

alcuna regola di business — quella vive nel dominio e nell'applicazione.

`store()` incatena due casi d'uso distinti, caricamento e avvio del workflow. `stream()` espone invece l'avanzamento della pipeline via Server-Sent Events: il controller resta responsabile solo del protocollo SSE (loop, heartbeat, timeout), leggendo lo stato tramite `PollDocumentProgressUseCase` invece di interrogare direttamente il database a ogni iterazione.

Attributi

- Nessuno.

Metodi

- `index(request: ListDocumentsRequest, list: ListDocumentsUseCase, state: MvpStateService): JsonResponse`
- `store(request: UploadDocumentRequest, upload: UploadDocumentUseCase, startWorkflow: StartDocumentWorkflowUseCase): JsonResponse`
- `stream(request: Request, originalDocument: OriginalDocument, state: MvpStateService, poll: PollDocumentProgressUseCase): StreamedResponse`
- `destroy(request: Request, subDocument: SubDocument, delete: DeleteDocumentUseCase, state: MvpStateService): JsonResponse`

5.1.3 DocumentPreviewController, SendMessageController

Due Controller separati da `DocumentController`, sullo stesso principio di traduzione pura verso una porta primaria: `DocumentPreviewController` espone la sola anteprima del file originale, `SendMessageController` l'intero ciclo del messaggio di invio (anteprima PDF composta, esportazione, correzione degli override). Entrambi traducono un fallimento applicativo in una risposta HTTP appropriata invece di lasciarlo propagare: `DocumentPreviewController` converte `DocumentPreviewUnavailableException` in un 404 e un `RuntimeException` generico (storage non raggiungibile) in un 503.

Attributi

- Nessuno.

Metodi

- `preview(request: Request, subDocument: SubDocument, preview: PreviewDocumentUseCase): Response`
- `sendPreview(request: Request, subDocument: SubDocument, sendMessage: SendMessageUseCase): Response`
- `sendExport(request: Request, subDocument: SubDocument, sendMessage: SendMessageUseCase): Response`
- `updateSendMessage(request: UpdateSendMessageRequest, subDocument: SubDocument, sendMessage: SendMessageUseCase, state: MvpStateService): JsonResponse`

5.1.4 DocumentWorkflowTaskHandler

Adapter primario guidato da Step Functions/SQS invece che da HTTP: sullo stesso piano concettuale del Controller, traduce ogni task di callback in arrivo dalla pipeline in una chiamata al caso d'uso corrispondente, tramite la sua porta primaria. Non contiene regole di business: l'idempotenza verso la ridelivery di un task (*documento già completato*) vive nei singoli casi d'uso, non in questo adapter.

Risolve e aggiorna l'aggregato documentale attraverso lo stesso `DocumentRepository` usato dai casi d'uso, mai attraverso Eloquent diretto. Comunica con il runner condiviso passando solo un identificativo e un tenant, così da restare agnostico rispetto al dominio.

Attributi

- `runOcr: RunOcrUseCase, processDocument: ProcessDocumentUseCase, finalize: FinalizeDocumentWorkflowUseCase, documents: DocumentRepository, clock: ClockInterface.`

Metodi

- `taskTypes(): array`
- `subjectType(): string`
- `resolveSubject(message: array): WorkflowSubject`
- `execute(taskType: string, subject: WorkflowSubject, message: array): array`
- `onFailure(subject: WorkflowSubject, taskType: string, e: Throwable): void`

5.1.5 OriginalDocument

Entità di dominio che rappresenta il documento caricato dall'utente e il suo avanzamento nella pipeline di elaborazione. Non è una semplice proiezione dei dati salvati: governa da sé le proprie transizioni di stato, invece di lasciare a ogni caso d'uso il compito di scriverle correttamente a mano.

Prima della sua introduzione, tre punti diversi del codice — l'avvio del workflow, il riconoscimento ottico, l'elaborazione AI — impostavano lo stato di fallimento con combinazioni di campi diverse e incoerenti fra loro; oggi `startProcessing()` e `fail()` consolidano quella logica in un unico posto, distinguendo sempre il messaggio pensato per l'operatore dal dettaglio tecnico usato per il troubleshooting. Non governa invece i campi restituiti dal riconoscimento ottico (testo, pagine, confidenza): non sono una transizione di stato, solo dati letti dal gateway OCR e scritti direttamente dal caso d'uso che li riceve.

Attributi

- `id, tenantId, filePath`, i metadati manuali di caricamento, `processingStatus`, il risultato OCR, i riferimenti all'esecuzione Step Functions.

Metodi

- `fromRecord(record: OriginalDocumentRecord): self`

- `startProcessing(executionArn: string, s3Bucket: string, s3Key: string, startedAt: DateTimeImmutable): void`
- `fail(errorMessage: string, technicalReason: ?string, failedAt: DateTimeImmutable): void`
- `complete(): void`
- `markWorkflowCompleted(completedAt: DateTimeImmutable): void`
- `pendingChanges(): OriginalDocumentChanges`

5.1.6 SubDocument

Entità di dominio che rappresenta un sotto-documento generato dallo split di un documento originale, e governa la propria revisione: può essere validato automaticamente dal sistema, messo in quarantena con un motivo esplicito quando l'estrazione ha una confidenza troppo bassa, oppure validato manualmente da un operatore.

Non governa invece lo stato di invio né gli eventuali override apportati al messaggio destinato al destinatario: quel flusso attraversa più aggregati contemporaneamente (il sotto-documento, i dati estratti, il documento originale) e non ha una collocazione naturale in una singola entità.

Attributi

- `id`, `originalDocumentId`, `filePath`, il range di pagine, `originalFilename`, `reviewStatus`.

Metodi

- `fromRecord(record: SubDocumentRecord, reviewStatus: ReviewStatus): self`
- `markAutoValidated(): void`
- `markNeedsReview(): void`
- `markQuarantined(reason: string): void`
- `markManuallyValidated(): void`
- `pendingChanges(): SubDocumentChanges`

5.1.7 UploadDocumentService

Implementa `UploadDocumentUseCase` ed è il punto di ingresso della pipeline documentale: riceve il file già validato dall'adapter HTTP, lo salva su storage e crea il record del documento originale con stato *Pending*. Deliberatamente non avvia la pipeline di elaborazione: quella è una decisione distinta, delegata a `StartDocumentWorkflowUseCase` e invocata in sequenza dallo stesso Controller.

L'audit non passa più da un `AuditLogger` chiamato direttamente: il caso d'uso dispatcha l'evento di dominio `DocumentUploadAccepted` (id documento, attore, nome file,

metadati manuali), registrato da un listener dedicato — stesso principio già in uso per il resto degli eventi di dominio.

Attributi

- `documents: DocumentRepository, storage: DocumentStoragePort, events: DocumentEventDispatcherPort.`

Metodi

- `upload(command: UploadDocumentCommand): int`

5.1.8 StartDocumentWorkflowService

Implementa `StartDocumentWorkflowUseCase`: avvia l'esecuzione Step Functions della pipeline per un documento già persistito, subito dopo `UploadDocumentService` nello stesso ciclo di richiesta. È idempotente rispetto a un documento già in elaborazione, ed esegue un controllo esplicito a runtime: quando Textract reale è abilitato, verifica che i documenti risiedano effettivamente su S3 reale e non sul disco LocalStack, fallendo subito con un messaggio azionabile invece di lasciare che Textract fallisca più a valle con un errore criptico. In caso di errore nell'avvio, marca il documento come fallito tramite `OriginalDocument::fail()` prima di rilanciare l'eccezione.

Non legge più configurazione tramite `config()` al proprio interno: ARN della state machine, URL della coda, flag Textract, disco e bucket di storage sono risolti una sola volta in `AppServiceProvider` e iniettati come scalari nel costruttore — lo stesso trattamento già riservato a soglia di confidenza e prefisso storage altrove nel dominio. Poiché anche `WorkflowContext` è una classe pura senza dipendenze da `Illuminate`, `start()` è interamente istanziabile ed eseguibile in un test Pest puro, senza bootstrap Laravel.

Attributi

- `documents: DocumentRepository, workflowEngine: WorkflowEnginePort, events: DocumentEventDispatcherPort, context: WorkflowContext, clock: ClockInterface, ids: UniqueIdGeneratorPort.`
- `stateMachineArn: string, taskQueueUrl: string, textractEnabled: bool, storageDisk: string, documentBucketFallback: string, documentKeyPrefix: string` — scalari risolti da configurazione in `AppServiceProvider`, non più letti internamente.

Metodi

- `start(documentId: int, correlationId: ?string, requestId: ?string): void`

5.1.9 RunOcrService

Implementa `RunOcrUseCase`: invoca il gateway OCR per il documento e ne salva il risultato grezzo (testo, pagine, confidenza media, id del job Textract) tramite `OriginalDocumentChanges`. In caso di errore marca il documento come fallito prima di rilanciare l'eccezione.

È idempotente verso la ridelivery del task workflow — un documento già completato restituisce immediatamente senza richiamare Textract — e deriva la chiave di idempotenza anche dall’oggetto S3, non solo dall’id del documento: dopo un reset seguito da un nuovo upload lo stesso id può puntare a un file diverso, e una chiave fissa farebbe scattare un conflitto contro il job precedente.

Attributi

- `documents: DocumentRepository, ocr: OcrGatewayPort, events: DocumentEventDispatcherPort, clock: ClockInterface.`

Metodi

- `run(documentId: int, bucket: ?string, key: ?string): array`

5.1.10 ProcessDocumentService

Coordina la fase più corposa della pipeline documentale: la classificazione del documento tramite Bedrock e, per ogni destinatario individuato, la delega dell’estrazione dei campi a `ExtractSubDocumentFieldsService`. Fino a una revisione recente era un’unica classe di quasi 480 righe, perché lo split del documento invocava internamente la logica di estrazione per ogni segmento appena creato; le due responsabilità sono state separate in classi distinte, mantenendo però la stessa forma di orchestrazione esterna.

Le sue dipendenze sono fra le più numerose del dominio, perché riflettono quante integrazioni coordina in un solo passo: un repository, uno storage, un gateway AI, un dispatcher di eventi, un heartbeat verso Step Functions — per segnalare che l’elaborazione è ancora viva durante chiamate AI che possono durare minuti — oltre a un generatore di identificativi e un lettore OCR condiviso.

Attributi

- `documents: DocumentRepository, storage: DocumentStoragePort, ai: DocumentAiGatewayPort, events: DocumentEventDispatcherPort, heartbeat: WorkflowHeartbeatPort, logger: LoggerInterface, ids: UniqueIdGeneratorPort, ocrRange: OcrRangeReader, fieldsExtractor: ExtractSubDocumentFieldsUseCase, clock: ClockInterface, tempDirectory: string.`

Metodi

- `process(documentId: int): array`

5.1.11 ExtractSubDocumentFieldsService

Implementa `ExtractSubDocumentFieldsUseCase`: estrae i campi strutturati (nominativo, azienda, data, tipo documento) di un singolo sotto-documento tramite Bedrock, un destinatario alla volta. Separata da `ProcessDocumentService` — prima di un refactor recente era un’unica classe di quasi 480 righe, perché lo split del documento invocava internamente l’estrazione per ogni segmento appena creato — la separazione isola questa logica dalla manipolazione PDF che la rendeva testabile solo insieme a quest’ultima.

I metadati dichiarati manualmente in fase di upload prevalgono sempre su quelli estratti dall'AI. Calcola un punteggio di confidenza combinando la confidenza media OCR sull'intervallo di pagine del sotto-documento con la completezza dei campi chiave effettivamente trovati, e ne deriva lo stato di revisione: sopra soglia il sotto-documento è validato automaticamente, altrimenti marcato per revisione manuale. Un output AI non conforme allo schema atteso mette il sotto-documento in quarantena invece di propagare l'errore.

Attributi

- `documents: DocumentRepository, ai: DocumentAiGatewayPort, events: DocumentEventDispatcherPort, logger: LoggerInterface, ocrRange: OcrRangeReader, tx: TransactionManagerPort, confidenceThreshold: int (default 80).`

Metodi

- `extractAndSaveFields(subDocumentId: int): void`
- `extractAndSaveFieldsWithContext(subDocumentId: int, original: OriginalDocument): void`

5.1.12 FinalizeDocumentWorkflowService

Implementa `FinalizeDocumentWorkflowUseCase`: chiude la pipeline documentale, marcando il workflow come completato sull'entità `OriginalDocument` quando lo stato di elaborazione è *Completed*, e pubblicando l'evento di completamento che la SPA osserva tramite lo stream SSE.

`currentStatus()` espone lo stato corrente per i punti della pipeline che devono decidere se procedere; `dispatchCompletionEvent()` distingue nel proprio risultato un completamento effettivo da un semplice avanzamento, così che il chiamante possa scegliere l'evento SSE corretto da inoltrare al client.

Attributi

- `documents: DocumentRepository, events: DocumentEventDispatcherPort, clock: ClockInterface.`

Metodi

- `currentStatus(documentId: int): string`
- `dispatchCompletionEvent(documentId: int): array`

5.1.13 PollDocumentProgressService

Implementa `PollDocumentProgressUseCase`: legge un'istantanea dello stato di avanzamento di un documento (stato di elaborazione, eventuale messaggio d'errore, identificativi dei sotto-documenti per cui l'estrazione è già disponibile), consumata da `DocumentController::stream()` a ogni iterazione del loop SSE invece di interrogare direttamente il repository dal controller.

Attributi

- `documents: DocumentRepository.`

Metodi

- `poll(documentId: int): DocumentProgressSnapshot`

5.1.14 ReviewDocumentService

Implementa `ReviewDocumentUseCase`: gestisce la revisione manuale di un sotto-documento da parte dell'operatore, con due metodi che coprono due percorsi distinti.

`updateExtractedData()` applica correzioni ai campi estratti e, in base al parametro esplicito ricevuto dal chiamante, marca il sotto-documento come validato manualmente oppure lo rimanda in revisione; scrive comunque i dati estratti se il sotto-documento non ne ha ancora, anche in assenza di modifiche. `markReviewed()` valida invece un sotto-documento senza modificarne i campi, e rifiuta l'operazione se non esistono ancora dati estratti da validare.

Attributi

- `documents: DocumentRepository, events: DocumentEventDispatcherPort.`

Metodi

- `updateExtractedData(subDocumentId: int, fieldUpdates: array, markAsValidated: bool, actor: ?Actor): string`
- `markReviewed(subDocumentId: int, actor: ?Actor): void`

5.1.15 PreviewDocumentService

Implementa `PreviewDocumentUseCase`: restituisce il contenuto grezzo del file di un sotto-documento per l'anteprima nella SPA, verificando prima che esista ancora sullo storage. Distinta da `SendMessageService::preview()`, che genera invece un PDF composto a partire dai dati estratti, non il file originale.

Verifica anche l'autorizzazione: se il tenant dell'attore non coincide con quello del documento originale, solleva `DocumentNotAuthorizedException` prima di leggere il file.

Attributi

- `documents: DocumentRepository, storage: DocumentStoragePort.`

Metodi

- `preview(subDocumentId: int, actor: Actor): PreviewableDocument`

5.1.16 SendMessageService

Implementa `SendMessageUseCase`: compone il messaggio di invio precompilato a partire dai dati già estratti — nessuna generazione AI coinvolta — e ne orchestra anteprima, esportazione e correzione manuale. Destinatario, oggetto e corpo hanno un valore proposto di default, sovrascrivibile dall'operatore tramite gli override salvati sul sotto-documento.

Il recapito avviene fuori dalla piattaforma: poiché il sistema non può osservare l'invio reale, è il download del PDF esportato a marcare lo stato come *Sent*, in una transizione a senso unico — un secondo download non la ripete. Tutti e tre i metodi pubblici verificano l'autorizzazione tramite un metodo privato condiviso (`assertActorOwnsSubDocument()`), che solleva `DocumentNotAuthorizedException` in caso di tenant non corrispondente.

Attributi

- `documents`: `DocumentRepository`, `renderer`: `SendMessageRendererPort`, `events`: `DocumentEventDispatcherPort`.

Metodi

- `preview(subDocumentId: int, actor: Actor): RenderedSendMessage`
- `export(subDocumentId: int, actor: Actor): RenderedSendMessage`
- `updateOverrides(subDocumentId: int, overrides: array, actor: Actor): void`

5.1.17 ListDocumentsService, DeleteDocumentService

Due casi d'uso minimi, simmetrici ai rispettivi sul dominio Communications. `ListDocumentsService` implementa `ListDocumentsUseCase` e inoltra la paginazione al repository tramite `DocumentListFilters` (value object tipizzato, non più un array grezzo), senza logica propria.

`DeleteDocumentService` implementa `DeleteDocumentUseCase`: verifica che l'attore appartenga allo stesso tenant del documento originale (`DocumentNotAuthorizedException` altrimenti), elimina un sotto-documento e, se non resta alcun altro sotto-documento collegato allo stesso documento originale, elimina anche quest'ultimo insieme ai suoi task di workflow pendenti. In entrambi i casi il record resta rimosso dal database anche se la pulizia dello storage fallisce: un guasto di rete verso S3 non deve far fallire un'eliminazione già avvenuta dal punto di vista dell'utente, a costo di lasciare temporaneamente un file orfano.

Attributi

- `ListDocumentsService`: `documents`: `DocumentRepository`.
- `DeleteDocumentService`: `documents`: `DocumentRepository`, `storage`: `DocumentStoragePort`, `events`: `DocumentEventDispatcherPort`, `logger`: `LoggerInterface`.

Metodi

- `list(tenantId: string, filters: DocumentListFilters, page: int, perPage: int): SubDocumentPage`
- `delete(subDocumentId: int, actor: Actor): void`

5.1.18 EloquentDocumentRepository

Adapter secondario che implementa `DocumentRepository` sopra Eloquent: è il punto in cui l'aggregato documentale (documento originale e sotto-documenti) viene effettivamente letto e scritto sul database. Consuma i value object tipizzati del dominio (`NewOriginalDocument`, `OriginalDocumentChanges`, `SubDocumentChanges...`) invece di array associativi grezzi, così che un refuso nel nome di una colonna venga intercettato dall'analisi statica invece che a runtime.

Attributi

- Nessuno.

Metodi

- `createOriginalDocument(document: NewOriginalDocument): int`
- `findOriginalDocument(id: int): OriginalDocument`
- `updateOriginalDocument(id: int, changes: OriginalDocumentChanges): void`
- `saveOriginalDocument(document: OriginalDocument): void`
- `deleteOriginalDocumentWithWorkflowTasks(id: int): void`
- `paginateSubDocuments(tenantId: string, filters: DocumentListFilters, page: int, perPage: int): SubDocumentPage`
- `findSubDocument(id: int): SubDocument`
- `saveSubDocument(subDocument: SubDocument): void`
- `createSubDocument(subDocument: NewSubDocument): int`
- `updateSubDocument(id: int, changes: SubDocumentChanges): void`
- `deleteSubDocument(id: int): void`
- `originalDocumentIdForSubDocument(subDocumentId: int): int`
- `originalDocumentHasRemainingSubDocuments(originalDocumentId: int): bool`
- `subDocumentIdsWithExtractedData(originalDocumentId: int): array`
- `deleteExistingSubDocuments(originalDocumentId: int): array` — legge ed elimina in un'unica transazione, restituendo i percorsi storage da ripulire.
- `saveExtractedData(subDocumentId: int, changes: ExtractedDataChanges): void`
- `deleteExtractedData(subDocumentId: int): void`

- `findSendMessageContext(subDocumentId: int): SendMessageContext`
- `subDocumentHasExtractedData(subDocumentId: int): bool`
- `countSubDocuments(originalDocumentId: int): int`

5.1.19 Adapter AI e OCR: `TextractOcrAdapter`, `BedrockDocumentAiAdapter`

I due adapter che collegano il dominio ai servizi AI esterni. `TextractOcrAdapter` implementa `OcrGatewayPort` sopra Amazon Textract e restituisce il risultato grezzo del riconoscimento ottico, senza toccare la persistenza: è il caso d'uso chiamante a decidere cosa salvarne.

`BedrockDocumentAiAdapter` implementa `DocumentAiGatewayPort` sopra il servizio condiviso `BedrockService` (Sezione 2.7), per lo split del documento e l'estrazione dei campi.

Attributi

- `TextractOcrAdapter`: `client: TextractClient`, `metrics: MetricsRecorder`, `heartbeat: WorkflowTaskHeartbeat`.
- `BedrockDocumentAiAdapter`: `bedrock: BedrockService`.

Metodi

- `detectText(bucket: string, key: string, idempotencyKey: string): array`
- `splitDocument(ocrText: string, pageCount: int, boundaryNonce: string): array`
- `extractFields(ocrText: string): array`

5.1.20 `FlysystemDocumentStorageAdapter`, `DompdfSendMessageRenderer`

Due adapter secondari di supporto. `FlysystemDocumentStorageAdapter` implementa `DocumentStoragePort` sopra il disco Laravel/Flysystem configurato per i documenti, generando per ogni file caricato un nome univoco (UUID) nella directory di destinazione, per evitare collisioni fra upload concorrenti.

`DompdfSendMessageRenderer` implementa `SendMessageRendererPort` sopra `Dompdf` e la vista Blade dedicata, applicando lo stesso timbro di piè di pagina (`PdfFooterStamper`) condiviso con l'esportazione delle comunicazioni.

Attributi

- `DompdfSendMessageRenderer`: `footerStamper: PdfFooterStamper`.
- `FlysystemDocumentStorageAdapter`: nessuno (il disco è risolto da configurazione a ogni chiamata).

Metodi

- `storeFromLocalPath(absoluteSourcePath: string, directory: string): string`
- `read(path: string): string, write(path: string, contents: string): void, delete(path: string): void, exists(path: string): bool`
- `renderPdf(composition: SendMessageComposition): string`

5.1.21 LaravelDocumentEventDispatcher

Adapter secondario minimo: implementa `DocumentEventDispatcherPort` appoggiandosi al dispatcher di eventi nativo di Laravel. I 14 listener del dominio (Sezione 4.4) sono registrati nel service provider, non in questa classe, che si limita a inoltrare l'evento.

Attributi

- `dispatcher: Dispatcher.`

Metodi

- `dispatch(event: object): void`

5.2 Modulo Communications (AI Assistant Generativo)

Il modulo Communications è la componente del sistema dedicata alla generazione assistita di comunicazioni interne, dal titolo al testo fino all'immagine di copertina. Il suo obiettivo è offrire ai redattori aziendali uno strumento che produca contenuti coerenti con il tono e lo stile dell'organizzazione a partire da un semplice prompt, riducendo il tempo dedicato alla scrittura senza sottrarre all'utente il controllo editoriale: ogni bozza generata resta modificabile, valutabile e rigenerabile prima dell'approvazione finale. Il modulo si integra con Amazon Bedrock sia per la generazione testuale sia per quella delle immagini di copertina, orchestrando la pipeline in modo asincrono tramite AWS Step Functions, sullo stesso schema del modulo Documents.

5.2.1 Diagramma di sequenza: generazione di una comunicazione

Il diagramma descrive la sequenza delle operazioni che avvengono quando un redattore richiede la generazione di una comunicazione tramite l'endpoint dedicato. Il `CommunicationController` riceve prompt, tono e stile, delega la creazione della bozza a `GenerateCommunicationService` e, subito dopo, l'avvio della pipeline a `StartCommunicationWorkflowUseCase`; la risposta HTTP torna al client con l'identificativo della comunicazione e l'URL di streaming, senza attendere la generazione.

Da quel momento la sequenza prosegue in modo asincrono. Il Worker riceve dalla state machine Step Functions un task alla volta: prima la generazione del testo, poi quella della copertina a partire dal prompt visivo prodotto dal passo precedente — un passo che può fallire senza invalidare la comunicazione, degradando semplicemente lo stato della copertina — infine la pubblicazione dell'evento di completamento.

In termini di classi, quei tre passi passano rispettivamente per `CommunicationWorkflowTaskHandler`, che delega a

`GenerateCommunicationTextService`, `GenerateCommunicationCoverService` e `FinalizeCommunicationUseCase`. Ogni passo aggiorna lo stato dell'entità `Communication` tramite `EloquentCommunicationRepository`.

In parallelo, la SPA resta connessa all'endpoint SSE esposto da `CommunicationStreamController`, che interroga `PollCommunicationProgressUseCase` a ogni iterazione e inoltra al client l'avanzamento di testo e copertina, fino all'evento finale di completamento o di errore.

5.2.2 CommunicationController

Adapter primario HTTP del dominio: raccoglie le operazioni sul ciclo di vita della bozza (creazione, modifica, rigenerazione, salvataggio, scarto, eliminazione, preferiti) che nel dominio Documents sono invece distribuite su più Controller specializzati, perché qui condividono lo stesso aggregato e la stessa autorizzazione di base.

Le operazioni più specifiche — copertina, esportazione, valutazione, streaming SSE — restano su Controller dedicati (`CommunicationCoverController`, `CommunicationExportController`, `CommunicationRatingController`, `CommunicationStreamController`), sullo stesso principio di traduzione pura verso una porta primaria, senza regole di business proprie.

Attributi

- Nessuno.

Metodi

- `index(request: ListCommunicationsRequest, list: ListCommunicationsUseCase, state: MvpStateService): JsonResponse`
- `store(request: GenerateCommunicationRequest, generate: GenerateCommunicationUseCase, startWorkflow: StartCommunicationWorkflowUseCase): JsonResponse`
- `update(request: UpdateCommunicationRequest, communication: Communication, draft: CommunicationDraftUseCase, state: MvpStateService): JsonResponse`
- `favorite(request: Request, communication: Communication, draft: CommunicationDraftUseCase, state: MvpStateService): JsonResponse`
- `unfavorite(request: Request, communication: Communication, draft: CommunicationDraftUseCase, state: MvpStateService): JsonResponse`
- `regenerate(request: Request, communication: Communication, startWorkflow: StartCommunicationWorkflowUseCase): JsonResponse`
- `save(request: Request, communication: Communication, draft: CommunicationDraftUseCase, state: MvpStateService): JsonResponse`
- `discard(request: Request, communication: Communication, draft: CommunicationDraftUseCase, state: MvpStateService): JsonResponse`

- `destroy(request: Request, communication: Communication, delete: DeleteCommunicationUseCase, state: MvpStateService): JsonResponse`

5.2.3 CommunicationWorkflowTaskHandler

Adapter primario guidato da Step Functions/SQS, equivalente concettuale del `DocumentWorkflowTaskHandler` sul dominio gemello: traduce i task di callback della pipeline di generazione in chiamate ai casi d'uso di dominio tramite le loro porte primarie, senza contenere regole di business proprie.

Fino a una revisione recente chiamava staticamente un metodo di formattazione errori sulla classe concreta `BedrockService` per costruire un messaggio di fallback; è stato allineato al comportamento del suo equivalente sul dominio Documents, che usa direttamente il messaggio dell'eccezione.

Attributi

- `generateText: GenerateCommunicationTextUseCase, generateCover: GenerateCommunicationCoverUseCase, finalize: FinalizeCommunicationUseCase, communications: CommunicationRepository, clock: ClockInterface.`

Metodi

- `taskTypes(): array`
- `subjectType(): string`
- `resolveSubject(message: array): WorkflowSubject`
- `execute(taskType: string, subject: WorkflowSubject, message: array): array`
- `onFailure(subject: WorkflowSubject, taskType: string, e: Throwable): void`

5.2.4 Communication

Entità di dominio che governa l'intero ciclo di vita di una bozza di comunicazione, ed è la più estesa dei due domini perché accumula responsabilità distinte — preferiti, approvazione e scarto della bozza, valutazione a stelle, copertina manuale o generata dall'AI, l'intera pipeline di generazione fino alla rigenerazione — invece delle una o due tipiche di `SubDocument` e `OriginalDocument`.

Assorbe anche un'invariante che in precedenza viveva in un value object a parte, oggi ritirato: una copertina non può mai essere applicata prima che il testo sia stato generato, verificato dalla stessa entità ogni volta che riceve l'esito della generazione AI. Come le altre due entità, ogni metodo verifica la propria guardia — una comunicazione già scartata non può essere approvata, una valutazione già assegnata non può essere ripetuta — e accumula il proprio delta in un oggetto di modifiche interno, letto una sola volta dall'adapter di persistenza.

Attributi

- `id`, `tenantId`, i dati della richiesta, l'output della generazione, gli stati di generazione/*copertina*/*bozza*, `isFavorite`, `rating`, i riferimenti all'esecuzione `Step Functions` ed eventuali errori.

Metodi

- `fromRecord(record: CommunicationRecord): self`
- `favorite(): void, unfavorite(): void`
- `updateDraft(title: string, body: string): void, approve(): void, discard(): void`
- `rate(rating: int, comment: ?string, actorId: string, ratedAt: DateTimeImmutable): void`
- `replaceCover(path: string, mime: string, size: int): void, removeCover(): void`
- `applyGeneratedText(generated: GeneratedCommunicationText): void, applyGeneratedCover(image: GeneratedCommunicationImage, path: string): void, degradeCover(warning: string): void`
- `startGeneration(executionArn: string, startedAt: DateTimeImmutable): void, failGeneration(errorMessage: string, technicalReason: ?string, failedAt: DateTimeImmutable): void, completeGeneration(completedAt: DateTimeImmutable): void`
- `regenerate(): void`
- `isReadyForExport(): bool, isEditable(): bool`
- `pendingChanges(): CommunicationChanges`

5.2.5 GenerateCommunicationService

Implementa `GenerateCommunicationUseCase` ed è il punto di ingresso della pipeline di generazione: crea la comunicazione con stato *Draft* e generazione *Pending* a partire da prompt, tono e stile, e restituisce l'identificativo appena creato. Come il suo equivalente sul dominio `Documents`, non avvia da sé la pipeline: quella responsabilità è delegata a `StartCommunicationWorkflowUseCase`, invocato subito dopo dallo stesso `Controller`.

L'audit non passa più da un `AuditLogger` diretto ma dal dispatch dell'evento `CommunicationGenerationRequested`, stesso principio già applicato a `UploadDocumentService` sul dominio `Documents`.

Attributi

- `communications: CommunicationRepository, events: CommunicationEventDispatcherPort.`

Metodi

- `generate(command: GenerateCommunicationCommand): int`

5.2.6 StartCommunicationWorkflowService

Implementa `StartCommunicationWorkflowUseCase`: avvia l'esecuzione Step Functions della pipeline di generazione, equivalente concettuale di `StartDocumentWorkflowService` sul dominio Documents, con la stessa idempotenza verso un'esecuzione già in corso. Espone anche `regenerate()`: riporta la comunicazione allo stato di rigenerazione, elimina l'eventuale copertina precedente dallo storage e riavvia la pipeline richiamando internamente `start()`, così che prima generazione e rigenerazione condividano sempre la stessa logica di avvio.

Stesso trattamento della configurazione del suo equivalente Documents: ARN della state machine e URL della coda non sono più letti tramite `config()` internamente, ma risolti una volta in `AppServiceProvider` e iniettati come scalari — stessa istanziabilità in un test Pest puro, senza bootstrap Laravel.

Attributi

- `communications`: `CommunicationRepository`, `coverStorage`: `CommunicationCoverStoragePort`, `workflowEngine`: `WorkflowEnginePort`, `events`: `CommunicationEventDispatcherPort`, `context`: `WorkflowContext`, `clock`: `ClockInterface`, `ids`: `UniqueIdGeneratorPort`.
- `stateMachineArn`: `string`, `taskQueueUrl`: `string` — scalari risolti da configurazione in `AppServiceProvider`.

Metodi

- `start(communicationId: int, correlationId: ?string, requestId: ?string): void`
- `regenerate(communicationId: int, actor: ?Actor, correlationId: ?string, requestId: ?string): void`

5.2.7 GenerateCommunicationTextService, GenerateCommunicationCoverService

Le due classi che orchestrano la generazione vera e propria, invocate dal Worker durante la pipeline asincrona: la prima genera titolo e corpo del testo tramite Bedrock, la seconda l'immagine di copertina a partire dal prompt visivo prodotto dalla prima. Sono rimaste distinte invece di essere unificate in un unico servizio, perché rappresentano due passi della state machine Step Functions con criticità diverse: se il testo non viene generato la bozza è fallita, mentre una copertina non disponibile viene degradata e segnalata, lasciando la comunicazione comunque valida.

Attributi

- `GenerateCommunicationTextService`: `communications`: `CommunicationRepository`, `ai`: `CommunicationAiGatewayPort`, `events`: `CommunicationEventDispatcherPort`.
- `GenerateCommunicationCoverService`: gli stessi tre più `storage`: `CommunicationCoverStoragePort`, `logger`: `LoggerInterface`, `ids`: `UniqueIdGeneratorPort`.

Metodi

- `generate(communicationId: int): array`

5.2.8 FinalizeCommunicationService

Implementa `FinalizeCommunicationUseCase`, equivalente concettuale di `FinalizeDocumentWorkflowService`: chiude la pipeline di generazione, marcando la comunicazione come completata e pubblicando l'evento di completamento osservato dallo stream SSE. È idempotente verso la ridelivery del task: una comunicazione già completata non ripubblica l'evento.

Se la copertina è rimasta bloccata in *Pending/Processing* — il ramo di degrado della state machine può saltare il task in caso di timeout o worker caduto — questo caso d'uso la degrada esplicitamente a *Failed* prima di chiudere la pipeline, così la SPA non resta in attesa indefinita.

Attributi

- `communications: CommunicationRepository, events: CommunicationEventDispatcherPort, clock: ClockInterface.`

Metodi

- `finalize(communicationId: int): array`

5.2.9 PollCommunicationProgressService

Implementa `PollCommunicationProgressUseCase`, equivalente concettuale di `PollDocumentProgressService`: legge un'istantanea dell'avanzamento (stato di generazione, stato copertina, testo generato finora, eventuali errori di copertina o di generazione), consumata da `CommunicationStreamController` a ogni iterazione dello stream SSE.

Attributi

- `communications: CommunicationRepository.`

Metodi

- `poll(communicationId: int): CommunicationProgressSnapshot`

5.2.10 CommunicationDraftService

Implementa `CommunicationDraftUseCase`: raccoglie le operazioni sul ciclo di vita editoriale della bozza — preferiti, modifica, approvazione, scarto — dietro cui il `CommunicationController` delega senza contenere logica propria. Ogni metodo segue lo stesso schema: recupera l'entità, invoca la transizione corrispondente (che verifica da sé le proprie guardie), persiste e pubblica l'evento di dominio associato.

`favorite()/unfavorite()` racchiudono lettura-per-aggiornamento e scrittura in una transazione tramite `TransactionManagerPort`, per evitare una corsa fra due toggle concorrenti sullo stesso preferito; `update()/save()/discard()` restano fuori transazione.

Attributi

- `communications: CommunicationRepository, events: CommunicationEventDispatcherPort, transactions: TransactionManagerPort.`

Metodi

- `favorite(communicationId: int, actor: Actor): void,`
`unfavorite(communicationId: int, actor: Actor): void`
- `update(communicationId: int, title: string, body: string, actor: Actor): void`
- `save(communicationId: int, actor: Actor): void,`
`discard(communicationId: int, actor: Actor): void`

5.2.11 RateCommunicationService

Implementa `RateCommunicationUseCase`: registra la valutazione a stelle di una comunicazione insieme a un commento opzionale, normalizzando un commento composto solo da spazi bianchi a `null` prima di passarlo all'entità. Lettura-per-aggiornamento, mutazione e salvataggio avvengono dentro una transazione (`TransactionManagerPort`), stessa cautela di `CommunicationDraftService::favorite()/unfavorite()` contro valutazioni concorrenti sulla stessa comunicazione.

Attributi

- `communications: CommunicationRepository, events: CommunicationEventDispatcherPort, clock: ClockInterface,`
`transactions: TransactionManagerPort.`

Metodi

- `rate(communicationId: int, rating: int, comment: ?string, actor: Actor): void`

5.2.12 UpdateCommunicationCoverService

Implementa `UpdateCommunicationCoverUseCase`: gestisce la sostituzione manuale della copertina da parte dell'utente, in alternativa a quella generata dall'AI. Rifiuta l'operazione se la comunicazione non è più modificabile (bozza già approvata o scartata).

Genera un percorso univoco per ogni nuova copertina caricata e rimuove quella precedente dallo storage solo dopo che la nuova è stata salvata con successo, per non lasciare la comunicazione temporaneamente senza copertina in caso di errore a metà operazione.

Attributi

- `communications: CommunicationRepository, storage: CommunicationCoverStoragePort.`
- `events: CommunicationEventDispatcherPort, ids: UniqueIdGeneratorPort, coverPathPrefix: string (default communications/covers).`

Metodi

- `update(communicationId: int, bytes: string, mime: string, size: int, actor: Actor): void`
- `remove(communicationId: int, actor: Actor): void`

5.2.13 DownloadCommunicationCoverService, ExportCommunicationService

Due casi d'uso di lettura per l'output finale di una comunicazione. `DownloadCommunicationCoverService` implementa `DownloadCommunicationCoverUseCase` e restituisce i byte grezzi della sola immagine di copertina.

`ExportCommunicationService` implementa `ExportCommunicationUseCase` e produce il PDF completo della comunicazione tramite `CommunicationPdfRendererPort`, rifiutando l'operazione se la bozza non è ancora pronta per l'esportazione (`isReadyForExport()`); espone anche un'impronta (`fingerprint()`) usata dall'adapter di rendering per la cache su disco, senza che il caso d'uso sappia che quella cache esiste.

Attributi

- `DownloadCommunicationCoverService: communications: CommunicationRepository, storage: CommunicationCoverStoragePort.`
- `ExportCommunicationService: communications: CommunicationRepository, renderer: CommunicationPdfRendererPort.`

Metodi

- `download(communicationId: int): DownloadableCoverImage`
- `fingerprint(communicationId: int): string, render(communicationId: int): RenderedCommunicationPdf`

5.2.14 ListCommunicationsService, DeleteCommunicationService

Due casi d'uso minimi, equivalenti concettuali di `ListDocumentsService` e `DeleteDocumentService`. `ListCommunicationsService` implementa `ListCommunicationsUseCase` e inoltra la paginazione al repository.

`DeleteCommunicationService` implementa `DeleteCommunicationUseCase`: elimina la comunicazione e, se presente, la sua copertina dallo storage — con lo stesso trattamento delle eliminazioni sul dominio Documents, un guasto nella pulizia dello storage viene solo loggato, non propagato, perché il record è già rimosso dal punto di vista dell'utente.

Attributi

- `ListCommunicationsService: communications: CommunicationRepository.`
- `DeleteCommunicationService: communications: CommunicationRepository, coverStorage: CommunicationCoverStoragePort, events: CommunicationEventDispatcherPort, logger: LoggerInterface.`

Metodi

- `list(tenantId: string, filters: CommunicationListFilters, page: int, perPage: int): CommunicationPage`
- `delete(communicationId: int, actor: Actor): void`

5.2.15 PromptConfigurationService

Implementa `PromptConfigurationUseCase`: gestisce le configurazioni di prompt salvate dall'utente (combinazioni di prompt, tono e stile riutilizzabili). Se il nome richiesto è vuoto o già in uso per il tenant, `save()` non restituisce un errore all'utente ma assegna un'etichetta progressiva ("*Senza nome (1)*", "*(2)*", ...) tramite un resolver privato, scegliendo sempre di lasciar salvare la configurazione piuttosto che bloccare l'utente su un conflitto di nome — invariato rispetto a prima.

Il nome è ora protetto anche da un vincolo di unicità a livello di database: `save()` tenta il salvataggio fino a 8 volte, e se l'adapter di persistenza segnala una collisione (`PromptConfigurationNameTakenException`, tradotta da una violazione del vincolo unico) rigenera un nome e riprova — una difesa contro la corsa fra due richieste concorrenti che il resolver, da solo, non può escludere; solo dopo 8 tentativi falliti l'eccezione risale al chiamante. `delete()` verifica ora l'autorizzazione: se il tenant della configurazione non coincide con quello dell'attore, solleva `PromptConfigurationNotAuthorizedException`.

Attributi

- `configurations: PromptConfigurationRepository, events: CommunicationEventDispatcherPort.`

Metodi

- `save(command: SavePromptConfigurationCommand): int`
- `delete(configurationId: int, actor: Actor): void`

5.2.16 EloquentCommunicationRepository

Adapter secondario che implementa `CommunicationRepository` sopra `Eloquent`, equivalente concettuale dell'`EloquentDocumentRepository`: consuma i value object tipizzati del dominio (`NewCommunication`, `CommunicationChanges`) invece di array grezzi.

È il punto di scrittura più consolidato di tutto il refactor verso l'architettura esagonale: prima della sua introduzione, lo stesso aggregato veniva scritto da diciotto punti diversi, distribuiti su otto casi d'uso.

Attributi

- Nessuno.

Metodi

- `createCommunication(communication: NewCommunication): int`
- `findCommunication(id: int): Communication`

- `findCommunicationForUpdate(id: int): Communication` — con `lockForUpdate()`, usato dai casi d'uso che racchiudono lettura e scrittura in una transazione.
- `updateCommunication(id: int, changes: CommunicationChanges): void`
- `saveCommunication(communication: Communication): void`
- `deleteCommunication(id: int): void`
- `paginateApprovedCommunications(tenantId: string, filters: CommunicationListFilters, page: int, perPage: int): CommunicationPage`

5.2.17 EloquentPromptConfigurationRepository

Adapter secondario che implementa `PromptConfigurationRepository` sopra `Eloquent`, per la persistenza delle configurazioni di prompt salvate dall'utente — un aggregato distinto da `Communication`, con un proprio repository invece di essere annegato in quello principale. `create()` riceve ora il value object tipizzato `NewPromptConfiguration` invece di un array grezzo, e traduce una violazione del vincolo unico a livello database in `PromptConfigurationNameTakenException`: è questa traduzione che `PromptConfigurationService::save()` intercetta nel proprio ciclo di retry.

Attributi

- Nessuno.

Metodi

- `nameExists(tenantId: string, name: string): bool`
- `create(configuration: NewPromptConfiguration): int`
- `tenantIdOf(id: int): string`
- `delete(id: int): void`

5.2.18 BedrockCommunicationAiAdapter

Adapter secondario che implementa `CommunicationAiGatewayPort` sopra il servizio condiviso `BedrockService` (Sezione 2.7), equivalente concettuale del `BedrockDocumentAiAdapter` sul dominio `Documents`: la generazione testuale e quella dell'immagine di copertina restano due chiamate distinte, coerentemente con il fatto che sono due passi separati della pipeline.

Attributi

- `bedrock: BedrockService.`

Metodi

- `generateText(prompt: string, tone: string, style: string): GeneratedCommunicationText`
- `generateImage(prompt: string, tone: string, style: string, modelImagePrompt: ?string): GeneratedCommunicationImage`

5.2.19 FlysystemCommunicationCoverAdapter, DompdfCommunicationPdfRenderer

Due adapter secondari di supporto, equivalenti concettuali di FlysystemDocumentStorageAdapter e DompdfSendMessageRenderer sul dominio Documents. FlysystemCommunicationCoverAdapter implementa CommunicationCoverStoragePort sopra Flysystem, registrando ogni fallimento di lettura/scrittura come metrica dedicata (communication_cover_storage_failed_total) oltre che nel log applicativo.

DompdfCommunicationPdfRenderer implementa CommunicationPdfRendererPort sopra Dompdf: prima di renderizzare calcola un'impronta SHA-1 del contenuto (titolo, corpo, stato e percorso della copertina) e, se un PDF con la stessa impronta è già stato materializzato su disco, lo restituisce direttamente invece di rigenerarlo; applica inoltre una filigrana diagonale (*"Creato da AI Assistant"*) e lo stesso timbro di piè di pagina condiviso con DompdfSendMessageRenderer.

Attributi

- FlysystemCommunicationCoverAdapter: metrics: MetricsRecorder.
- DompdfCommunicationPdfRenderer: footerStamper: PdfFooterStamper.

Metodi

- store(path: string, bytes: string): void, read(path: string): string, delete(path: string): void, exists(path: string): bool
- fingerprint(context: CommunicationPdfContext): string
- render(context: CommunicationPdfContext): string, filename(context: CommunicationPdfContext): string

5.2.20 LaravelCommunicationEventDispatcher

Adapter secondario minimo, equivalente concettuale del LaravelDocumentEventDispatcher: implementa CommunicationEventDispatcherPort appoggiandosi al dispatcher di eventi nativo di Laravel. I 21 listener del dominio (Sezione 4.4) sono registrati nel service provider.

Attributi

- dispatcher: Dispatcher.

Metodi

- dispatch(event: object): void

5.3 Classi condivise

Infrastruttura trasversale ai due domini, non riorganizzata in porte e adapter propri (si veda la Sezione 4.1.1 per la motivazione del perimetro).

- **WorkflowTaskRegistry** e **WorkflowTaskRunner** (**Workflow/Services**): instradano un task in arrivo da Step Functions/SQS verso l'handler del dominio competente, gestendo dedup, claim e audit in modo generico.
- **WorkflowTaskHeartbeat** (**Workflow/Services**): unico adapter condiviso della porta **WorkflowHeartbeatPort**, invocata dai casi d'uso di lunga durata (split PDF, OCR) per segnalare a Step Functions che l'elaborazione è ancora viva, senza che quei casi d'uso conoscano **SfnClient**.
- **SfnWorkflowEngineAdapter** (**Workflow/Adapters/Outbound**): unico adapter che implementa la porta condivisa verso Step Functions per entrambi i domini.
- **WorkflowContext** (**Workflow/Support**): contenitore puro degli identificativi di correlazione (request id, correlation id, tenant) propagati lungo l'esecuzione di un caso d'uso, senza alcuna dipendenza da Illuminate — il tagging dei log (**Log::withContext()**) non vive più qui, ma nel middleware **HTTP CorrelateRequests** e, per il worker SQS, in **ConsumeWorkflowTasks**.
- **Actor** (**Support/Identity**): chi ha compiuto un'azione (id, tenant, ruoli), tradotto una sola volta al bordo HTTP da **MvpUser**; porte, eventi e comandi di dominio vedono solo questa classe, mai il contratto **Authenticatable** di Laravel.
- **MvpUserProvider** (**Identity**): **UserProvider** esplicito per il guard **mvp**, registrato in **AppServiceProvider::boot()**. **MvpUser** non ha un archivio persistente — viene ricostruito a ogni richiesta da **ResolveMvpIdentity** e impostato con **Auth::setUser()**, mai risolto da un provider — quindi ogni metodo di questa classe fallisce deliberatamente con un messaggio esplicito invece di lasciare che una risoluzione inattesa (es. un **Auth::check()** eseguito prima del middleware) fallisca con un errore Eloquent criptico, come accadeva prima quando **config/auth.php** dichiarava un driver **eloquent** per un modello che non estende **Model**.
- **SystemClock** (**Support/Clock**): unico adapter condiviso dell'interfaccia standard **PSR-20 ClockInterface**, usata al posto di un helper temporale custom.
- **RandomUuidGenerator** (**Support/Identifiers**): unico adapter condiviso della porta **UniqueIdGeneratorPort**, per la generazione di identificativi univoci.
- **LaravelTransactionManager** (**Support/Persistence**): unico adapter condiviso della porta **TransactionManagerPort**, per le sequenze di scritture DB pure che richiedono un confine transazionale.
- **AuditLogger** (**Audit/Services**): registra gli eventi di audit dispatchati dai listener di dominio.
- **MetricsRecorder** e **PrometheusExporter** (**Observability**): raccolgono ed espongono le metriche applicative sull'endpoint **/internal/metrics**, descritto nella Sezione 2.9.

6 Architettura Frontend

6.1 Principi di progettazione

La SPA Angular è organizzata per feature (`features/assistant`, `features/copilot`, `features/overview`), affiancate da tre cartelle trasversali: `core` (stato applicativo, interceptor HTTP, osservabilità), `shared` (componenti UI riutilizzabili) e `layout` (guscio dell'applicazione, intestazioni, navigazione). Ogni feature raggruppa a sua volta una pagina (View), il proprio ViewModel, i propri componenti (`components/`) e i propri servizi/modelli (`data/`), rispecchiando la stessa separazione per dominio applicativo già adottata nel backend (Sezione 4.3).

Tutti i componenti sono standalone (nessun `NgModule`) e usano `ChangeDetectionStrategy.OnPush` con le API a segnali di Angular 21 (`signal`, `computed`, `input()`) al posto delle proprietà decorate con `@Input()`: un componente si ridisegna solo quando un segnale che consuma cambia valore, non a ogni ciclo di change detection globale. Il routing e i template usano la sintassi di controllo nativa (`@if`, `@for`) introdotta nelle versioni recenti del framework, senza le direttive strutturali `*ngIf`/`*ngFor`. Le tre pagine principali (`AssistantPage`, `CopilotPage`, `OverviewPage`) separano inoltre la logica di presentazione dalla vista tramite un ViewModel dedicato (Sezione 6.2), un principio applicato solo lì: i componenti di feature e quelli condivisi restano invece componenti Angular ordinari, senza un ViewModel proprio.

6.2 Il pattern MVVM

Le tre pagine principali applicano MVVM in senso classico: un *ViewModel* — più precisamente un *Presentation Model* nel senso di Fowler — è una classe TypeScript che non tocca il DOM né l'infrastruttura di rendering o di dependency injection di Angular (nessun `@Component`, `inject()`, decoratore, injection context, chiamata diretta a `document/window`); riceve le proprie dipendenze dal costruttore, ed è quindi istanziabile con un semplice `new` e testabile senza `TestBed`, con dipendenze mockate a mano. L'unica eccezione apparente è l'uso di `signal/computed` di `@angular/core`: restano nel ViewModel non come parte del motore di rendering, ma come primitiva reattiva di piattaforma — l'equivalente di `INotifyPropertyChanged` in WPF — quindi non compromettono l'istanziabilità con `new` né la testabilità senza Angular avviato; è questa distinzione, non un conteggio letterale degli import da `@angular/core`, a rendere la classe un Presentation Model genuino.

Il *Model* resta quanto descritto in Sezione 6.3: i tipi generati dal contratto OpenAPI e i modelli di dominio specifici di ciascuna feature. La *View* è la pagina stessa (`AssistantPage`, `CopilotPage`, `OverviewPage`): l'unico punto accoppiato ad Angular e al DOM, che si procura le dipendenze con `inject()` nel proprio costruttore e le passa al ViewModel per costruirlo, poi lega il template ai segnali e ai metodi pubblici che il ViewModel espone (`vm.stato()`, `(evento)="vm.metodo()"`). Anche l'unica operazione DOM richiesta da alcuni ViewModel — lo scroll verso una sezione dopo un'azione — resta un'operazione della View: `AssistantPageViewModel` e `OverviewPageViewModel` la ricevono come funzione dal costruttore (`scrollToElement`, definita in `shared/util/scroll.ts`), così possono *chiedere* uno scroll senza *eseguirlo* direttamente.

Lo stato condiviso fra le pagine non vive nel ViewModel: una sola istanza di `MvpStateStore`, fornita a livello di applicazione (`providedIn: "root"`), conserva lo stato restituito dal backend e sopravvive ai cambi di rotta. Ogni ViewModel riceve lo Store come una qualunque altra dipendenza nel proprio costruttore e lo consuma in sola lettura per derivare i propri segnali `computed`; le mutazioni (generazione, upload, revisione, eliminazione) non aggiornano lo stato localmente, ma rimpiazzano l'intero stato con quello autorevole restituito dal backend nella risposta, evitando divergenze fra ciò che la SPA mostra e ciò che il backend ha effettivamente persistito.

Un'eccezione è documentata esplicitamente nel codice: `effect()` e `takeUntilDestroyed()` restano nella View perché richiedono un injection context che il ViewModel non ha per costruzione. L'`effect()` però si limita a leggere i segnali sorgente (lo storico condiviso, i filtri attivi) e a chiamare `vm.reload()`: la chiamata di ricerca vera e propria — stessa forma di ogni altra azione del ViewModel, come `generate()` o `discard()` — vive nel ViewModel, non nella View. I metodi che un tempo la View chiamava direttamente per inoltrare il risultato (`setFilteredCommunications()`, `handleHistoryError()`) sono ora privati: dettagli interni di `reload()`, non più superficie pubblica del ViewModel. Per lo stesso motivo il template non legge mai `store.*` direttamente: anche `error`, `loading` e le metriche sono esposti come segnali `computed` pass-through sul ViewModel (`vm.error()`, `vm.loading()`, `vm.metrics()`), che si limita a leggerli dallo Store già ricevuto come dipendenza.

Angular offre già disaccoppiamento e testabilità tramite i propri servizi e la dependency injection, quindi un ViewModel separato potrebbe sembrare ridondante rispetto a testare direttamente il componente con `TestBed`. Il motivo per cui si è scelto comunque questo confine è pratico, non dogmatico: un test su `TestBed` richiede il bootstrap del modulo, la compilazione del template e la risoluzione dell'albero di dependency injection a ogni esecuzione, mentre un test sul ViewModel istanzia la classe con `new` passando dipendenze mockate a mano — più veloce da eseguire e più semplice da scrivere, perché isola la logica applicativa dal ciclo di vita di Angular. Per questo il confine è tracciato solo sulle tre pagine principali, dove quella logica è più densa, e non esteso a ogni componente della SPA.

L'assenza di `inject()` e dell'infrastruttura di rendering nel ViewModel non è un'omissione stilistica: è la condizione che rende possibile l'obiettivo dichiarato sopra. `inject()` funziona solo dentro un injection context, quindi una classe che lo usasse sarebbe istanziabile solo da Angular — per testarla servirebbe comunque `TestBed.runInInjectionContext()` o un `TestBed.inject()`, lo stesso costo che l'estrazione del ViewModel vuole evitare. Lo stesso vale per il rendering: un ViewModel che manipolasse il DOM richiederebbe un ambiente browser (o `jsdom`) anche solo per verificarne la logica. Estendere questo pattern a ogni componente della SPA, non solo alle tre pagine principali, non è però la conclusione corretta: il beneficio (test framework-indipendenti su logica applicativa densa) va confrontato con il costo (un file e un'indirection in più). I dodici componenti condivisi (Sezione 6.6) non hanno quasi logica propria — ricevono un `input()` e lo mostrano — e un ViewModel dedicato aggiungerebbe solo un file vuoto da mantenere, senza nulla di significativo da isolare che il codice non mostri già direttamente.

6.2.1 Esempio nel progetto: AssistantPageViewModel

Il ViewModel più corposo, perché copre l'intero ciclo di vita di una bozza (generazione, rigenerazione, valutazione, copertina, storico, configurazioni di prompt salvate). Il test unitario associato (`assistant-page.view-model.spec.ts`) lo costruisce con `new` passando un `AssistantService`, un `MvpStateStore` e una funzione di scroll mockati a mano — nessun bootstrap Angular — a riprova diretta che qui MVVM non è solo nominale. `CopilotPageViewModel` e `OverviewPageViewModel` seguono la stessa forma per le rispettive pagine, con una superficie più piccola; `CopilotPageViewModel` non riceve la dipendenza di scroll perché non ne ha mai avuto bisogno.

Attributi

- `assistant: AssistantService`, `store: MvpStateStore`, `scrollTo: (elementId: string) => void` — dipendenze ricevute dal costruttore, non da `inject()`; `scrollTo` è `scrollToElement` passata dalla View.
- `phase`, `isGenerating`, `latestDraft`, `previewDraft`, `status` — segnali sullo stato della generazione in corso e della bozza visualizzata.
- `error: Signal<string | null>` — pass-through su `store.error()`, così il template non legge mai lo Store direttamente.
- `activeFilters`, `filteredCommunications`, `historyError` — segnali sullo storico, letti dal template; scritti solo internamente da `reload()`.

Metodi

- `generate(payload: CommunicationDraftForm): void`, `regenerate(): void`
- `rateDraft(payload: RateDraftPayload): void`,
`selectDraft(communicationId: number): void`
- `uploadCover(file: File): void`, `removeCover(): void`
- `saveDraft(event: {communicationId: number, payload: UpdateCommunicationRequest}): void`
- `discard(): void`, `saveToHistory(): void`,
`deleteHistoryItem(communicationId: number): void`
- `toggleFavorite(communication: Communication): void`
- `saveConfiguration(payload: SavePromptConfigurationRequest): void`,
`useConfiguration(configuration: PromptConfiguration): void`,
`deleteConfiguration(configurationId: number): void`
- `setActiveFilters(filters: CommunicationFilters): void`

- **reload(): void** — rilegge lo storico con i filtri correnti; stessa forma delle altre azioni, chiamata dall'**effect()** della View a ogni cambio di filtro o di stato condiviso. Annulla la propria sottoscrizione precedente prima di avviarne una nuova (fix di una regressione: senza questa cancellazione, una risposta di ricerca arrivata in ritardo poteva sovrascrivere un risultato più recente).
- **destroy(): void** — chiamato dalla View alla distruzione del componente (tramite **DestroyRef**): annulla solo la sottoscrizione di lettura in corso, non le azioni di scrittura (**generate()**, **discard()**...), che devono completare lato server anche se l'utente ha già navigato altrove.

6.2.2 Esempio nel progetto: **MvpStateStore** (stato condiviso fra i **ViewModel**)

Sorgente unica dello stato applicativo (Assistant e Co-Pilot), condivisa da tutte le viste tramite i rispettivi **ViewModel**. Espone segnali di sola lettura derivati da un unico segnale privato, così che né i **ViewModel** né i componenti possano scrivere lo stato direttamente, ma solo tramite i metodi dedicati.

Attributi

- **state: Signal<MvpState | null>**, **loading: Signal<boolean>**, **error: Signal<string | null>** — segnali di sola lettura.
- **documents**, **history**, **promptConfigurations**, **assistantMetrics**, **copilotMetrics** — segnali derivati (**computed**) sulle diverse viste dello stato.

Metodi

- **loadOnce(): void** — carica lo stato al primo montaggio della shell applicativa; a differenza di prima non usa più un flag "già richiesto" a senso unico, ma deriva la guardia dallo stato reale dei segnali (**loading** o **state** già valorizzato), così un primo tentativo fallito può essere ritentato da una chiamata successiva invece di restare bloccato per sempre.
- **reload(): void** — ricarica lo stato dal backend, con un tentativo di retry per errori temporanei.
- **setState(state: MvpState): void** — rimpiazza lo stato con quello autorevole restituito da una mutazione.
- **upsertDocument(document: SubDocument): void** — inserisce o aggiorna in testa un sotto-documento ricevuto dallo stream SSE, senza ricaricare l'intero stato.
- **metric(key: string): number** — legge una metrica per chiave stabile, evitando di ricalcolarla su elenchi parzialmente caricati.

6.3 Definizione dei modelli di dati

Il livello Model si articola su due strati. Il primo è generato automaticamente da Orval a partire dal contratto OpenAPI (Sezione 2.4): interfacce TypeScript per ogni schema (`Communication`, `SubDocument`, `MvpState`...), non modificabili a mano. Il secondo è specifico di ciascuna feature e arricchisce i tipi generati con concetti che esistono solo lato UI — fasi di una pipeline, etichette, vincoli dei form — senza duplicarne i campi.

6.3.1 Esempio nel progetto: `assistant.model.ts`

Arricchisce i tipi generati per il modulo Assistant. `CommunicationGenerationPhase` traduce le combinazioni di `generationStatus/coverStatus` restituite dal backend in un'unica fase discreta, usata dal template per etichette e stato dei controlli senza replicare quella logica in più punti; `GeneratedDraft` è la proiezione della bozza usata dal pannello di anteprima, con solo i campi che quella vista consuma.

Attributi

- `CommunicationDraftForm` — alias del tipo di richiesta generato, usato dal form di generazione.
- `CommunicationGenerationPhase` — unione di stringhe letterali (*queued*, *generating-text*, *generating-cover*, *still_running*, *completed*, *failed*).
- `CommunicationGenerationProgress`, `GeneratedDraft`, `RateDraftPayload` — interfacce di proiezione per, rispettivamente, l'avanzamento SSE, l'anteprima e la valutazione.
- `communicationTones`, `communicationStyles` — le opzioni di tono e stile selezionabili, condivise fra form e validazione.

6.3.2 Esempio nel progetto: tipi del modulo Co-Pilot

L'equivalente `Co-Pilot` di `assistant.model.ts`, distribuito fra `document-workflow.service.ts` (i tipi) e `document-filters.ts` (il form dei filtri e la sua conversione). `DocumentUploadPhase` traduce l'evento SSE `progress` in una fase discreta per la barra di avanzamento, analogamente a `CommunicationGenerationPhase` sul modulo Assistant. `toDocumentFilters()` converte i valori grezzi del form nei criteri inviati all'API scartando quelli vuoti, evitando percorsi che possano sollevare eccezioni: un valore inatteso nel form non deve poter interrompere silenziosamente lo stream dei filtri, come segnalato nel commento della funzione che genera il form.

Attributi

- `DocumentUploadPhase` — unione di stringhe letterali (*uploading*, *queued*, *processing*, *extracting*, *completed*, *failed*).
- `DocumentFilters`, `ConfidenceCriterion`, `DocumentUploadMetadata`, `DocumentUploadProgress`, `DocumentPreviewStatus` — interfacce e unioni per, rispettivamente, i criteri di ricerca, il confronto di soglia, i metadati di upload, l'avanzamento e lo stato dell'anteprima PDF.

- `DocumentFilterFormValue` — tipo derivato dal `FormGroup` dei filtri tramite `ReturnType`, così i due restano sincronizzati senza doverli dichiarare due volte.

Metodi

- `createDocumentFilterForm(): FormGroup`
- `toDocumentFilters(value: DocumentFilterFormValue): DocumentFilters`

6.4 Servizi di comunicazione con il backend

Ogni feature ha un servizio dedicato (`AssistantService`, `DocumentWorkflowService`) che incapsula le chiamate al client generato e aggiorna lo Store con la risposta autorevole. Nessun componente né `ViewModel` chiama direttamente il client HTTP: passa sempre dal servizio della propria feature, ricevuto come dipendenza dal costruttore del `ViewModel` (Sezione 6.2).

6.4.1 Esempio nel progetto: `AssistantService`

Il servizio più corposo del frontend, perché incapsula anche il consumo dello stream SSE lato client, simmetrico a quello che il backend espone (Sezione 3.2). `trackGeneration()` avvolge la chiamata HTTP iniziale e la connessione SSE risultante in un unico `Observable`: chi lo sottoscrive riceve una sequenza di stati di avanzamento (*queued* → testo → copertina → *completed/failed*) senza dover gestire direttamente gli eventi SSE.

La connessione non usa più `EventSource` direttamente: la meccanica di basso livello (apertura dello stream, parsing dei frame, distinzione fra evento nominato `error` e caduta della connessione) è stata estratta in `SseClient` (Sezione 6.4, sottosezione successiva); questo servizio resta responsabile solo dell'orchestrazione specifica di feature (interpretazione degli eventi *progress/text/cover/done*, aggiornamento dello Store).

Attributi

- `api: AlittlebyteMVPAPIService, store: MvpStateStore, sse: SseClient.`

Metodi

- `generate(payload: CommunicationDraftForm): Observable<CommunicationGenerationProgress>`
- `regenerate(communicationId: number): Observable<CommunicationGenerationProgress>`
- `searchCommunications(filters: CommunicationFilters): Observable<Communication[]>`
- `rate(communicationId: number, payload: RateCommunicationRequest): Observable<RateCommunicationResponse>`
- `favorite(communicationId: number): Observable<UpdateCommunicationFavoriteResponse>`
- `unfavorite(communicationId: number): Observable<UpdateCommunicationFavoriteResponse>`

- `updateCoverImage(communicationId: number, image: File): Observable<CommunicationMutationResponse>`
- `removeCoverImage(communicationId: number): Observable<CommunicationMutationResponse>`
- `saveToHistory(communicationId: number): Observable<CommunicationMutationResponse>`
- `discard(communicationId: number): Observable<CommunicationMutationResponse>`
- `deleteFromHistory(communicationId: number): Observable<DeleteDocumentResponse>`
- `saveConfiguration(payload: SavePromptConfigurationRequest): Observable<SavePromptConfigurationResponse>`
- `deleteConfiguration(configurationId: number): Observable<DeleteDocumentResponse>`

6.4.2 Esempio nel progetto: DocumentWorkflowService

Equivalente Co-Pilot di `AssistantService`, con lo stesso incapsulamento dello stream SSE lato client. `upload()` avvolge la chiamata HTTP di caricamento e la connessione SSE risultante in un unico `Observable`, esattamente come `trackGeneration()` — anche qui la meccanica di basso livello è delegata a `SseClient` (sottosezione successiva), non più a un `EventSource` costruito direttamente in questa classe.

`previewStatus()` è invece un caso specifico di questo modulo, assente sul dominio `Communications` e indipendente da SSE: verifica il *content-type* della risposta prima di montare l'iframe di anteprima, perché l'endpoint può restituire il PDF (200), un 404 se il file non è ancora disponibile o un 503 JSON se lo storage non è raggiungibile — tre esiti che il componente chiamante deve distinguere per mostrare lo stato corretto.

Attributi

- `api: ALittleByteMVPAPIService, store: MvpStateStore, sse: SseClient.`

Metodi

- `upload(file: File, metadata: DocumentUploadMetadata): Observable<DocumentUploadProgress>`
- `deleteSubDocument(documentId: string): Observable<DeleteDocumentResponse>`
- `saveExtractedData(documentId: string, payload: UpdateExtractedDataRequest): Observable<UpdateSubDocumentReviewResponse>`
- `markReviewed(documentId: string): Observable<UpdateSubDocumentReviewResponse>`

- `saveSendMessage(documentId: string, payload: UpdateSendMessageRequest): Observable<UpdateSubDocumentReviewResponse>`
- `searchDocuments(filters: DocumentFilters): Observable<SubDocument[]>`
- `previewStatus(previewUrl: string): Observable<DocumentPreviewStatus>`

6.4.3 Esempio nel progetto: SseClient

Client SSE condiviso da entrambi i servizi di feature, iniettabile a livello applicazione (`providedIn: "root"`). A differenza di `EventSource`, è basato su `fetch`: può quindi inviare gli header di correlazione (`X-Request-Id`, `X-Correlation-Id`) sulla richiesta iniziale di apertura dello stream, cosa che `EventSource` non permette.

Distingue esplicitamente l'evento SSE nominato `error` (dati applicativi, es. un messaggio di errore) dalla caduta della connessione o da una risposta non riuscita — due condizioni che `EventSource` collassa in un unico `onerror`, costringendo il chiamante a indovinare quale sia successo.

Attributi

- Nessuno: nessuno stato d'istanza, ogni chiamata a `connect()` è indipendente.

Metodi

- `connect(url: string, handlers: SseHandlers): () => void` — apre lo stream e restituisce una funzione di chiusura (basata su `AbortController`), da invocare alla disiscrizione.

6.5 Serializzazione e deserializzazione dei dati

A differenza di un backend Rails con serializer scritti a mano per ogni risorsa, qui la serializzazione verso e dal backend è interamente generata: Orval crea, insieme alle interfacce dei modelli, anche le funzioni client che serializzano il corpo delle richieste e deserializzano le risposte JSON secondo i tipi del contratto OpenAPI. Non esiste quindi un adapter di serializzazione scritto a mano da documentare: la conformità fra client e contratto è verificata automaticamente in CI (Sezione 2.10, `openapi:check`), che rigenera il client e fallisce se il risultato differisce da quello committato.

6.6 Micro componenti UI

I componenti condivisi (`shared/components`) sono privi di logica di dominio: ricevono dati tramite segnali di input e rimettono eventi tramite `output()`, senza conoscere da quale feature vengono usati. Sono dodici in tutto: `ButtonComponent`, `AlertComponent`, `ErrorStateComponent`, `EmptyStateComponent`, `LoadingStateComponent`, `StatusBadgeComponent`, `MetricCardComponent`, `MetricsPanelComponent`, `ProgressBarComponent`, `SelectFieldComponent`, `TextAreaFieldComponent`, `FileDropzoneComponent`. La cartella `data-table` contiene solo un foglio di stile condiviso, non un componente: le tabelle restano markup HTML nativo nelle singole pagine, non un elemento riutilizzabile a sé.

6.6.1 Esempio nel progetto: ButtonComponent

Il più semplice dei componenti condivisi: non introduce un nuovo elemento HTML, ma si applica come attributo su un `<button>` nativo (`<button mvpButton variant="secondary">`). In questo modo il componente lascia al browser tipo, stato disabilitato, attributi ARIA ed eventi nativi, occupandosi solo della variante di stile applicata all'host.

Attributi

- **variant:** `InputSignal<"primary" | "secondary" | "icon">` — variante di stile, di default *primary*.
- **type:** `InputSignal<string>` — tipo dell'attributo HTML `type`, di default *button*.

Metodi

- Nessuno: il comportamento è interamente dichiarativo, espresso nei binding `host` del decoratore `@Component`.

6.6.2 AlertComponent, ErrorStateComponent

`AlertComponent` è il riquadro di base per un messaggio con titolo, usato ovunque serva segnalare qualcosa con enfasi (`role="alert"`, `aria-live="polite"`).

`ErrorStateComponent` non duplica quel markup: lo compone, fissando un titolo di default (*"Servizio non disponibile"*) e passando il messaggio d'errore come contenuto proiettato — è il componente che compare in cima a ogni pagina quando lo stato di errore è valorizzato. Espone ora anche un pulsante di ripetizione opzionale (*"Riprova"*), renderizzato solo quando `canRetry()` è vero: alla pressione emette l'evento `retry`, lasciando al chiamante decidere cosa ritentare.

Attributi

- **AlertComponent: title:** `InputSignal<string>` (obbligatorio).
- **ErrorStateComponent: message:** `InputSignal<string>` (obbligatorio), **title:** `InputSignal<string>` (default *"Servizio non disponibile"*), **canRetry:** `InputSignal<boolean>` (default *false*).

Metodi

- **ErrorStateComponent: retry:** `OutputEmitterRef<void>` — emesso al click sul pulsante "Riprova", visibile solo se `canRetry()` è vero.
- **AlertComponent:** nessuno.

6.6.3 EmptyStateComponent, LoadingStateComponent, StatusBadgeComponent

Tre componenti minimi di visualizzazione, senza alcuna logica: si limitano a proiettare contenuto o un'etichetta dentro un elemento con lo stile e la semantica ARIA corretti (`role="status"` per il caricamento, colori diversi per tono sull'etichetta di stato).

Attributi

- **EmptyStateComponent**: `nessuno`, solo `<ng-content>`.
- **LoadingStateComponent**: `label`: `InputSignal<string>` (default *"Caricamento in corso."*).
- **StatusBadgeComponent**: `tone`: `InputSignal<StatusTone>` (*info, success, warning, danger*, default *neutral*).

Metodi

- Nessuno.

6.6.4 MetricCardComponent, MetricsPanelComponent

MetricCardComponent mostra una singola coppia valore/etichetta. **MetricsPanelComponent** la compone in una griglia, gestendo da sé i tre stati della richiesta a monte: caricamento (**LoadingStateComponent**), elenco vuoto (**EmptyStateComponent**) o dati presenti — così ogni pannello di metriche della SPA (Overview, Assistant, Co-Pilot) ottiene lo stesso comportamento senza ripeterlo.

Attributi

- **MetricCardComponent**: `value`: `InputSignal<string | number>` (obbligatorio), `label`: `InputSignal<string>` (obbligatorio).
- **MetricsPanelComponent**: `isLoading`: `InputSignal<boolean>` (default *false*), `metrics`: `InputSignal<Metric[]>` (default `[]`).

Metodi

- Nessuno.

6.6.5 ProgressBarComponent

Barra di avanzamento orizzontale condivisa fra le pipeline asincrone di entrambi i moduli: non conosce le fasi di dominio (*extracting, generating-cover...*), riceve già una percentuale e uno stato visivo generico, ed è ogni feature a mappare la propria fase su questi ultimi. Finché lo stato è *active* un bagliore scorre da sinistra a destra; con **prefers-reduced-motion** resta fermo, per conformità WCAG 2.3.3.

Attributi

- `value`: `InputSignal<number>` (obbligatorio), `state`: `InputSignal<ProgressState>` (*idle, active, done, error*), `label`: `InputSignal<string>`.
- `clamped`: `Signal<number>` — segnale derivato che vincola `value` fra 0 e 100 prima di applicarlo alla larghezza della barra.

Metodi

- Nessuno.

6.6.6 SelectFieldComponent, TextAreaFieldComponent

I due wrapper di campo condivisi dai form reattivi della SPA, con la stessa struttura: un'etichetta associata via `<label for>`, un identificativo generato automaticamente dal testo dell'etichetta se non fornito esplicitamente, e un `FormControl` ricevuto come segnale di input invece che tramite `ControlValueAccessor` — una scelta più semplice, sufficiente perché nessuno dei due deve integrarsi con `ngModel`.

Attributi

- **SelectFieldComponent:** `label`, `options`: `InputSignal<readonly string[]>`, `control`: `InputSignal<FormControl<string>>` (tutti obbligatori), `id`: `InputSignal<string>` (opzionale).
- **TextAreaFieldComponent:** `label`, `control` (obbligatori), `rows`: `InputSignal<number>` (default 4), `placeholder`, `id` (opzionali).

Metodi

- Nessuno.

6.6.7 FileDropzoneComponent

Area di trascinalamento/selezione file per il caricamento documenti nel Co-Pilot, ristretta ai soli PDF (`accept="application/pdf"`). Ripulisce il valore dell'`<input>` subito dopo aver emesso il file selezionato, così lo stesso file può essere ricaricato una seconda volta senza che il browser lo consideri un valore invariato e sopprima l'evento `change`.

Attributi

- `label`: `InputSignal<string>` (obbligatorio), `disabled`: `InputSignal<boolean>` (default *false*).

Metodi

- `fileSelected`: `OutputEmitterRef<File>` — evento emesso alla selezione di un file valido.

6.7 Macro componenti UI (pagine)

Le pagine sono i componenti radice di ciascuna feature, montati dal router: `OverviewPage` (cruscotto con le metriche di entrambi i moduli), `AssistantPage` (AI Assistant Generativo) e `CopilotPage` (AI Co-Pilot). Nel pattern MVVM descritto in Sezione 6.2 sono la View: si procurano con `inject()` le dipendenze Angular (Store, servizio di feature) e costruiscono il proprio `ViewModel` passandogliele nel costruttore, poi compongono i componenti di feature e quelli condivisi legando il template ai segnali e ai metodi esposti dal `ViewModel`. Sulla View restano solo gli aspetti che richiedono un injection context Angular che il `ViewModel` non ha — i `FormGroup` reattivi dei filtri e l'`effect()` che li collega al `ViewModel` — oltre a un'unica azione locale priva di logica di dominio (`resetFilters()`); nessuna chiamata HTTP e nessuna regola di business vive più nella pagina.

6.7.1 Esempio nel progetto: AssistantPage

Componi il pannello di generazione (`CommunicationGeneratorPanelComponent`) e quello di anteprima (`GeneratedCommunicationPreviewComponent`), entrambi componenti di feature, insieme a componenti condivisi per stato di errore e stato vuoto, legando quasi ogni input e ogni evento del template a un segnale o un metodo di `vm` (`AssistantPageViewModel`, Sezione 6.2). L'unica eccezione deliberata è il banner d'errore principale: `[canRetry]="true" (retry)="store.reload()"` chiama direttamente lo Store, non il ViewModel, perché un errore di caricamento a livello di pagina è un problema dello stato condiviso, non della logica specifica di questa feature.

Nel costruttore istanzia il ViewModel con `new AssistantPageViewModel(inject(AssistantService), store, scrollToElement)` — passandogli anche l'unica funzione che tocca il DOM, così il ViewModel resta un Presentation Model puro — e collega un `FormGroup` di filtri e un `effect()` che si limita a leggere lo storico condiviso e i filtri attivi del ViewModel, poi chiama `vm.reload()`: la chiamata di ricerca vera e propria vive nel ViewModel, non qui. Registra anche `inject(DestroyRef).onDestroy(() => vm.destroy())`, così la sottoscrizione di lettura in corso viene annullata alla distruzione del componente senza che il ViewModel sappia nulla del ciclo di vita di Angular.

Attributi

- **store:** `MvpStateStore` — dipendenza Angular procurata con `inject()`, campo `protected` (non più `private`): il template la legge direttamente solo per il `retry` del banner d'errore principale (`store.reload()`), oltre a passarla al costruttore del ViewModel e leggerla come segnale sorgente nell'`effect()`.
- **vm:** `AssistantPageViewModel` — istanziato nel costruttore, punto a cui si lega il resto del template.
- **filterForm:** `FormGroup` — form reattivo dei filtri dello storico, resta sulla View perché `ReactiveFormsModule` richiede un `InjectionContext`.

Metodi

- **resetFilters():** `void` — unico metodo proprio della View, oltre alla logica di collegamento nel costruttore.

6.7.2 Esempio nel progetto: CopilotPage

Stessa struttura di `AssistantPage`, applicata al Co-Pilot: componi il pannello di upload (`DocumentUploadPanelComponent`), l'elenco documenti (`DocumentListComponent`) e il pannello di revisione del sotto-documento selezionato (`SubDocumentListComponent`), leggendo dal template anche `vm.error()`, `vm.loading()` e `vm.metrics()` invece dei rispettivi segnali sullo Store — con la stessa unica eccezione di `AssistantPage`: il banner d'errore principale usa `[canRetry]="true" (retry)="store.reload()"` direttamente sullo Store. Istanzia il ViewModel con `new CopilotPageViewModel(inject(DocumentWorkflowService), store)` — senza la dipendenza di scroll, perché questo ViewModel non ne ha mai avuto bisogno (Sezione 6.2) — e collega allo stesso modo un `FormGroup` di filtri (`createDocumentFilterForm()`) e un

`effect()` che rilegge lo storico documentale chiamando `vm.reload()` a ogni cambio di filtro o di stato condiviso. Registra allo stesso modo `inject(DestroyRef).onDestroy(() => vm.destroy())`.

Attributi

- `store`: `MvpStateStore` — dipendenza Angular procurata con `inject()`, campo `protected`: stesso ruolo e stessa eccezione (retry del banner d'errore) che ha su `AssistantPage`.
- `vm`: `CopilotPageViewModel` — istanziato nel costruttore.
- `filterForm`: `FormGroup` — form reattivo dei filtri dello storico documenti.

Metodi

- `resetFilters()`: `void` — unico metodo proprio della View.

6.7.3 Esempio nel progetto: `OverviewPage`

La pagina più semplice: nessun `FormGroup`, nessun `effect()`, perché la `Overview` non ha uno storico filtrabile proprio — mostra solo le metriche aggregate e le attività recenti già presenti nello Store. Compone componenti condivisi (`MetricCardComponent`, `MetricsPanelComponent`, `StatusBadgeComponent`, `EmptyStateComponent`) senza alcun componente di feature proprio.

Istanzia il ViewModel con `new OverviewPageViewModel(store, router)`; i pulsanti di richiamo all'azione nella sezione hero delegano interamente a `vm.navigate()`, che naviga e poi chiede lo scroll verso la sezione target sulla pagina di destinazione. Il banner d'errore usa `[canRetry]="true" (retry)="store.reload()"`, esattamente come in `AssistantPage/CopilotPage`: qui però è l'unico punto in cui la View legge lo Store direttamente, non un'eccezione a un pattern altrimenti diverso, perché questa pagina non ha un ViewModel con proprio stato di errore.

Attributi

- `store`: `MvpStateStore` — dipendenza Angular procurata con `inject()`, usata sia dalla View (per `store.error()/store.loading()` e per il `retry`, `store.reload()`) sia passata al ViewModel.
- `vm`: `OverviewPageViewModel` — istanziato nel costruttore insieme a `inject(Router)`.

Metodi

- Nessuno: a differenza delle altre due pagine, non ha bisogno di cablare filtri o effetti reattivi propri.

7 Design Pattern

I design pattern individuati nel sistema si dividono in due categorie: quelli applicati esplicitamente nella logica applicativa, con un adapter, una porta o un collaboratore reale

dietro il nome — non citati per completare l'elenco — e i pattern nativi del framework, adottati così come sono senza reimplementarli. Per ciascun pattern applicato esplicitamente viene indicato dove si trova nel codice, quale problema risolve in quel punto specifico e cosa succederebbe senza. Due pattern valutati e scartati (Proxy, Abstract Factory) sono documentati alla fine della Sezione 7.1, per lo stesso motivo: un pattern citato solo per apparire nell'elenco non è meno fuorviante di uno assente.

Pattern	Categoria	Dove applicato
Adapter	Strutturale	I repository Eloquent, gli adapter Bedrock/Texttract e <code>SfnWorkflowEngineAdapter</code> di entrambi i domini (Sezione 5)
Strategy	Comportamentale	<code>WorkflowTaskHandler</code> (porta primaria), selezionato da <code>WorkflowTaskRegistry::for(\$taskType)</code>
Facade	Strutturale	I casi d'uso applicativi (<code>UploadDocumentService</code> , <code>GenerateCommunicationService</code> , ...), un solo metodo pubblico per coordinare più porte secondarie
Factory Method	Creazionale	<code>WorkflowTaskRegistry::for(string \$taskType): WorkflowTaskHandler</code>
Singleton	Creazionale	Binding dei client AWS e degli adapter nel service provider, bindati all'interfaccia di porta
Observer	Comportamentale	Porte dispatcher di eventi dei due domini → dispatcher Laravel, 14 + 21 listener
Command	Comportamentale	Ogni caso d'uso applicativo, una classe per una porta primaria

Tabella 16: Design pattern individuati nel backend

7.1 Design Pattern Backend

7.1.1 Adapter

Intent: convertire l'interfaccia di una classe in un'altra interfaccia attesa dal client, permettendo la collaborazione fra classi altrimenti incompatibili per via delle librerie o dei framework esterni che usano.

È il pattern costitutivo dell'intera architettura esagonale (Sezione 4.3.2): isola sistematicamente il dominio applicativo dalla forma specifica delle tecnologie esterne — ORM, SDK cloud — che altrimenti imporrebbero la propria interfaccia direttamente ai casi d'uso. Ogni porta secondaria ha un adapter che la implementa sopra una tecnologia concreta, mai il contrario.

Esempio nel progetto. `EloquentDocumentRepository` ed `EloquentCommunicationRepository` implementano le porte di persistenza sopra Eloquent, `BedrockDocumentAiAdapter`, `BedrockCommunicationAiAdapter` e `TexttractOcrAdapter` sopra gli SDK AWS, `SfnWorkflowEngineAdapter` sopra Step Functions (Sezione 5). Un caso d'uso parla sempre con `DocumentRepository`, mai con `SubDocument::query()`. Senza questo pattern il dominio dipenderebbe direttamente da Eloquent e dagli SDK AWS, e nessun caso d'uso sarebbe testabile senza bootstrap Laravel e credenziali AWS/LocalStack — la Sezione 4.7 quantifica il numero di test di dominio resi possibili da questa scelta.

7.1.2 Strategy

Intent: definire una famiglia di algoritmi intercambiabili, incapsularli singolarmente e renderli intercambiabili indipendentemente dal client che li usa.

Risolve il problema di un componente infrastrutturale condiviso da più domini che deve restare agnostico rispetto al passo di business specifico di ciascuno, delegando solo quel passo a un'implementazione selezionata a runtime invece di conoscerla in anticipo.

Esempio nel progetto. `WorkflowTaskHandler` è la porta primaria implementata da `DocumentWorkflowTaskHandler` e `CommunicationWorkflowTaskHandler` (Sezione 5): `WorkflowTaskRunner`, il collaboratore condiviso che riceve un task da Step Functions/SQS, resta completamente agnostico rispetto al dominio — dedup, claim, audit e metriche sono identici per entrambi — e delega il solo passo di business all'handler selezionato da `WorkflowTaskRegistry` in base al tipo di task. Senza questo pattern, `WorkflowTaskRunner` avrebbe bisogno di un `match/if` sul dominio del task, violando la Dependency Rule verificata in CI (Sezione 4.7): l'infrastruttura di workflow condivisa dovrebbe conoscere i dettagli interni di Documents e Communications.

7.1.3 Facade

Intent: fornire un'interfaccia unificata a un insieme di interfacce di un sottosistema, semplificando l'uso del sottosistema per il client.

Evita che l'adapter primario — un Controller HTTP o un handler di workflow — debba conoscere e orchestrare direttamente i singoli collaboratori necessari a soddisfare una richiesta applicativa, riducendo quella conoscenza a un solo metodo pubblico.

Esempio nel progetto. Ogni caso d'uso applicativo (Sezione 5) è una Facade sulle proprie porte secondarie: `ProcessDocumentService` coordina repository, storage, gateway AI, dispatcher di eventi, heartbeat e generatore di identificativi dietro un solo metodo pubblico (`process()`); l'adapter primario che lo invoca — `DocumentController` o `DocumentWorkflowTaskHandler` — non deve sapere quanti collaboratori servono per soddisfare la richiesta. Senza questo pattern il Controller orchestrerebbe direttamente 4-5 servizi, la situazione esplicitamente evitata dalla separazione fra adapter primario e caso d'uso applicativo su cui si fonda tutta la Sezione 5.

7.1.4 Factory Method

Intent: definire un'interfaccia per creare un oggetto, lasciando che sia un metodo dedicato a decidere quale classe concreta istanziare, invece del chiamante.

Centralizza in un solo punto la selezione dell'implementazione corretta a runtime, così che il chiamante non debba conoscere in anticipo l'insieme delle classi concrete disponibili né duplicare la logica di scelta a ogni punto di invocazione.

Esempio nel progetto. `WorkflowTaskRegistry::for(string $taskType): WorkflowTaskHandler` seleziona l'handler corretto per tipo di task, senza che `WorkflowTaskRunner` (il chiamante) conosca le classi concrete `DocumentWorkflowTaskHandler/CommunicationWorkflowTaskHandler`. Senza questo pattern, quella selezione andrebbe duplicata con logica propria a ogni punto di invocazione del runner.

7.1.5 Singleton

Intent: garantire che una classe abbia una sola istanza condivisa e fornire un punto di accesso globale ad essa.

Evita la costruzione ripetuta di collaboratori costosi da istanziare — un client HTTP verso un servizio esterno, tipicamente — e centralizza in un solo punto la scelta di quale implementazione concreta soddisfa una data porta.

Esempio nel progetto. I client AWS (`SfnClient`, il client Bedrock, il client Textract) e i rispettivi adapter sono bindati come singleton nel service provider — costruzione costosa evitata, connessioni riutilizzate — ma bindati all'interfaccia di porta, non alla classe concreta: il binding diventa così anche il punto unico in cui si decide quale adapter soddisfa quale porta. Senza questa scelta, cambiare adapter richiederebbe cercare e modificare ogni type-hint concreto sparso nella codebase.

7.1.6 Observer

Intent: definire una dipendenza uno-a-molti fra oggetti, così che quando un oggetto cambia stato tutti i suoi dipendenti vengano notificati automaticamente, senza che il soggetto li conosca singolarmente.

Disaccoppia l'esecuzione di una regola di business dalle sue conseguenze collaterali — audit, metriche — che diventano reazioni indipendenti a un evento invece di chiamate esplicite ripetute in ogni caso d'uso che ne ha bisogno.

Esempio nel progetto. Applicato simmetricamente a entrambi i domini, con porte separate perché gli eventi sono specifici di dominio: Documents dispatcha 14 eventi (`DocumentEventDispatcherPort` → `LaravelDocumentEventDispatcher`, 14 listener), Communications 21 (`CommunicationEventDispatcherPort` → `LaravelCommunicationEventDispatcher`, 21 listener) — si veda la Sezione 5 per l'elenco delle classi che li dispatchano. Prima dell'adozione sistematica del pattern, le chiamate ad audit e metriche erano sparse manualmente in ogni caso d'uso: la coppia audit+metrica per "copertina degradata" era duplicata in due punti distinti (degrado da errore del modello o dello storage, degrado da timeout in `FinalizeCommunicationService`) — con l'evento unico quella duplicazione sparisce. Senza Observer, ogni nuova reazione a un evento di dominio richiederebbe toccare ogni caso d'uso che genera quell'evento, invece di aggiungere un listener.

7.1.7 Command

Intent: incapsulare una richiesta come oggetto a sé stante, permettendo di parametrizzare i client con richieste diverse e di trattare ogni richiesta come una singola unità testabile in isolamento.

Trasforma ogni operazione applicativa in un oggetto con una singola responsabilità, testabile passando mock delle sue dipendenze senza dover avviare l'intero framework che lo circonda.

Esempio nel progetto. Ogni caso d'uso applicativo (Sezione 5) è una classe dedicata a una singola porta primaria, la stessa forma già usata da `WorkflowTaskHandler::execute()`: un caso d'uso, una responsabilità, testabile passando mock delle porte secondarie senza bootstrap Laravel. Nota di onestà: l'idea "un

caso d'uso = una classe con un solo metodo pubblico" non regge alla lettera per le porte che raggruppano più transizioni sullo stesso aggregato — `CommunicationDraftUseCase` ne ha cinque (`favorite/unfavorite/update/save/discard`), `StartCommunicationWorkflowUseCase` e `UpdateCommunicationCoverUseCase` ne hanno due ciascuna. È una scelta esplicita — raggruppare transizioni correlate sullo stesso aggregato invece di una classe per transizione — non un Command puro: si applica a metà dei casi d'uso, non a tutti.

7.1.8 Pattern nativi del framework

Laravel contribuisce due pattern propri, adottati così come sono anziché reimplementati: Eloquent (variante di *Active Record*, POEAA) per il mapping oggetti-tabelle, usato però solo dentro gli adapter di persistenza, mai nel dominio — è proprio ciò che l'Adapter (Sezione 7.1.1) isola. Il Service Container di Laravel fornisce la dependency injection su cui si appoggiano sia il binding a Singleton (Sezione 7.1.5) sia la risoluzione automatica dei costruttori dei casi d'uso. Da non confondere con l'omonimo pattern Facade di Laravel (`Log::`, `Storage::`, `Auth::` — un accesso statico ai binding del container, un'accezione diversa dal GoF Facade applicato ai casi d'uso in Sezione 7.1.3): quelle Facade restano infrastruttura ai margini del sistema, mai usate dentro Documents o Communications — lo stesso script di CI che verifica la Dependency Rule (Sezione 4.7) lo impedirebbe.

Due pattern sono stati valutati e scartati esplicitamente, non per assenza di un candidato ma perché inapplicabili al problema reale: **Proxy** (caching o circuit-breaking davanti ai gateway Bedrock/Texttract) contraddirebbe una scelta esplicita del progetto: nessun fallimento di un servizio AI deve essere mascherato da un comportamento sostitutivo automatico. **Abstract Factory** non ha un candidato reale: non esiste nel progetto una famiglia di adapter correlati da costruire coerentemente insieme — gli adapter per LocalStack e per AWS reale non sono famiglie diverse, sono la stessa classe con endpoint e credenziali diversi — e l'unico bisogno di selezione a runtime è già coperto dal Factory Method di `WorkflowTaskRegistry`.

7.2 Design Pattern Frontend

7.2.1 Facade

Intent: fornire un'interfaccia unificata a un insieme di interfacce di un sottosistema, semplificando l'uso del sottosistema per il client — lo stesso intento del Facade backend (Sezione 7.1.3), applicato al lato client.

Evita che un ViewModel debba orchestrare direttamente più collaboratori del livello dati (client HTTP, canale SSE, stato condiviso) per compiere una singola azione, riducendo quell'orchestrazione a un solo metodo.

Esempio nel progetto. `AssistantService` e `DocumentWorkflowService` (Sezione 6.4) sono Facade sul client HTTP generato, su `SseClient` e sullo Store condiviso: un ViewModel chiama un solo metodo (`generate()`, `upload()`...) senza sapere che dietro c'è una richiesta HTTP, l'apertura di uno stream SSE e un aggiornamento dello stato applicativo. Senza questo pattern, ogni ViewModel dovrebbe orchestrare direttamente client API, `SseClient` e `MvpStateStore` per ciascuna azione, duplicando la stessa sequenza di passi in più punti.

7.2.2 Presentation Model

Intent (Fowler, POEAA): rappresentare lo stato e il comportamento della view in una classe indipendente dai widget della piattaforma, così che la view sia una proiezione sottile di quello stato invece di contenerlo.

A differenza del Facade, non è un pattern GoF ma architetturale (POEAA, come Repository o Active Record). Separa la logica applicativa di una pagina dal framework che la ospita, rendendola verificabile senza avviare l'infrastruttura di rendering — al prezzo di un file in più per ogni pagina a cui viene applicato.

Esempio nel progetto. Descritto per intero in Sezione 6.2: ogni pagina principale (`AssistantPage`, `CopilotPage`, `OverviewPage`) ha un `*ViewModel` sibling — classe TypeScript pura, costruita con `new`, senza `inject()` né riferimenti al DOM — che il Component costruisce e a cui lega l'intero template. Il minimo richiesto sarebbe il `@Component` stesso come `ViewModel` (MVVM in senso lasco): scartato come unica implementazione perché non regge a un test costruito con `new`, il requisito esplicito del refactor. Senza questo pattern, ogni test sulla logica di una pagina richiederebbe `TestBed/Injector.create()` — il costo che l'intera Sezione 6.2 spiega di voler evitare, e solo dove il beneficio lo giustifica (le tre pagine principali, non ogni componente della SPA).